

BCC WORKSHOP

Bringing Machine Learning into Production

Modul lengkap dari Step 0 sampai Step 10

Studi kasus: NYC Yellow Taxi Demand Forecasting

Ahmad Nafi Mubarak
Fatoni Murfid Syafii

Setup - Data - Training - DVC - MLflow - Airflow
API - Docker - CI/CD - Monitoring - Retraining

Edisi 7 Oktober 2026
Repository: [bcc-workshop-mlops](#)
Revision number: 51c9f5d

Daftar isi

Panduan membaca	3
Pengantar - Memahami MLOps	4
Studi kasus - NYC Yellow Taxi	10
Step 0 - Setup environment	14
Step 1 - Data dan historical replay	18
Step 2 - Feature dan baseline model	26
Step 3 - Snapshot dan DVC	33
Step 4 - MLflow dan model registry	40
Step 5 - Airflow orchestration	48
Step 6 - Serving melalui FastAPI	59
Step 7 - Docker dan Compose	66
Step 8 - CI/CD dan container delivery	73
Step 9 - Prometheus, Grafana, dan evaluasi	82
Step 10 - Retraining dan promotion	95
Lampiran A - Quickstart dashboard dari nol	108
Lampiran B - Troubleshooting	113
Lampiran C - Glosarium	117
Lampiran D - Laporan dan panduan fasilitator	122

Panduan membaca

Semua materi utama tersedia di satu dokumen ini: konsep awal, persiapan environment, data, feature, baseline, DVC, MLflow, Airflow, serving, Docker, CI/CD, monitoring, serta retraining. Ikuti Step 0 sampai Step 10 secara berurutan. Lampiran berisi quickstart, troubleshooting, glosarium, dan laporan akhir.

Cara menggunakan modul

- Jalankan command dari root repository melalui Bash di Linux, macOS, atau WSL2.
- Ganti placeholder RUN_ID atau RUN_ID_MODEL dengan run ID hasil training. Ganti VERSION atau VERSI_MODEL dengan nomor version dari registry; pertahankan tanda kutip pada command Bash.
- Angka metric dan contoh model version dalam materi adalah ilustrasi, kecuali kamu sendiri menghasilkan nilainya.
- Selesaikan checkpoint sebelum berpindah tahap. Halaman service yang terbuka belum membuktikan data atau model sudah tersedia.
- Pilih satu instance Airflow. Riwayat standalone dan Docker Compose terpisah.
- Catat run ID, dataset SHA256, model version, dan hasil evaluasi untuk laporan akhir.

Port dan mode runtime

Service	Default repo	Catatan
Docs	8002	Supaya tidak bentrok dengan prediction API
MLflow	5000	Host menggunakan localhost; container menggunakan hostname mlflow
FastAPI	8000	Swagger tersedia pada /docs
Airflow	8080	Ikuti host port pada docker compose ps airflow; contoh 18081 di materi adalah pemetaan alternatif
Prometheus	9090	Periksa target taxi-api pada /targets
Grafana	3000	Dashboard Taxi Demand MLOps

Catatan untuk command yang panjang

Command inspeksi python -c yang panjang ditulis ulang sebagai blok uv run python - <<'PY' agar dapat dibaca dan disalin dengan utuh. Jalankan seluruh blok termasuk penutup PY. Tanda backslash pada command Bash lain menandakan kelanjutan baris.

Kesiapan workshop

Peserta cukup mengenal Python, Pandas, regression, train/validation split, dan Git dasar. Download dependency, image, serta data TLC dapat memakan waktu; siapkan sebelum sesi kelas. Rancangan live workshop sekitar 6 jam termasuk jeda.

Pengantar - Memahami MLOps

Kalau kalian search “What is MLOps?” di internet, sering banget ketemu kalimat:

MLOps is DevOps for Machine Learning.

Arah definisinya nggak salah. Tapi buat pemula, itu kadang belum membantu karena malah muncul pertanyaan baru: “DevOps aja belum terlalu ngerti, terus kenapa ML perlu versi khusus?” Jadi kita mulai dari problem, bukan definisi.

Project ML paling sederhana

Misalnya kalian punya:

```
project/  
├─ notebook.ipynb  
├─ data.csv  
└─ model.pkl
```

Di notebook, kalian load data, train, evaluate, lalu save model. Result:

```
MAE = 10.2
```

Untuk experimentation, itu fine. Problem baru muncul ketika model berubah dari:

“Eksperimen saya.”

menjadi:

“Something other people depend on.”

Begitu ada dependency ke orang atau system lain, requirement-nya ikut berubah.

Problem pertama: reproducibility

Tiga minggu kemudian kalian buka folder data.

```
data.csv  
data_final.csv  
data_fix.csv  
data_fix_bener.csv  
data_final_bener_2.csv
```

Relatable? Model yang MAE 10.2 tadi pakai yang mana? Terus ternyata feature engineering pernah direvisi. Versi scikit-learn beda. Parameter pernah diganti. Random seed mungkin beda. Jadi training context itu bukan cuma code.

```
Code  
+  
Data  
+  
Parameters  
+  
Environment  
+  
Randomness
```

Kalau satu berubah, result bisa berubah. Makanya kita butuh beberapa layer:

```

Git
→ code history

DVC
→ training data snapshot

MLflow
→ experiment context

uv
→ environment consistency

```

Perhatikan bahwa tools ini saling melengkapi, bukan saling menggantikan.

Problem kedua: deployability

Kalian punya file model. Backend developer tanya:

"Cara aplikasi gue request prediction gimana?"

Kalau jawabannya:

"Import file Python saya, load model, terus bikin lag feature manual."

Technically bisa, tapi coupling-nya tinggi banget. Backend jadi harus tahu model framework, feature order, lag logic, rolling window, model path, dan historical data. Lebih clean kalau kita punya contract:

```

http
POST /predict

```

Request:

```

json
{
  "zone_id": 161,
  "target_datetime": "2025-01-28T18:00:00"
}

```

Response:

```

json
{
  "predicted_trip_count": 147.8
}

```

Client nggak perlu tahu internal model implementation. Itu abstraction boundary.

Problem ketiga: observability

Sekarang API hidup. HTTP 200. Latency 80 ms. Apakah sistem sehat? Belum tentu.

Case A - API sehat, model jelek

```

HTTP 200
latency 60 ms
zero server error

```

Tapi prediction meleset jauh terus-menerus. Operationally sehat. Model-wise buruk.

Case B - model bagus, service bermasalah

Model accuracy masih bagus, tapi API latency 15 detik dan request timeout. Model-wise sehat. Service-wise buruk. Jadi ML system punya minimal dua health dimension:

```
System Health
+
Model Health
```

Ini konsep yang harus benar-benar kalian pegang.

Problem keempat: maintainability

Data dunia nyata berubah. Behavior berubah. Seasonality berubah. Business context berubah. Contoh taxi:

```
Training:
January

Later:
February / March
different traffic
different event
different demand pattern
```

Model yang bagus di January belum tentu bagus selamanya. Jadi lifecycle production bukan:

```
train
↓
deploy
↓
finish
```

lebih realistis:

```
train
↓
deploy
↓
observe
↓
evaluate
↓
retrain if needed
↓
review
↓
promote
↓
observe again
```

Itu loop.

Jadi MLOps itu apa?

Sekarang definisinya lebih meaningful. Di workshop ini kita pakai definisi practical:

MLOps adalah practices dan engineering yang membuat lifecycle Machine Learning menjadi reproducible, deployable, observable, dan maintainable.

Empat kata ini penting:

```
reproducible  
deployable  
observable  
maintainable
```

Kalau mau ingat MLOps, ingat empat itu.

Kenapa ML beda dari software biasa?

Traditional software behavior mostly ditentukan code. Machine Learning beda. Behavior model dipengaruhi:

```
code  
data  
features  
parameters  
library version  
random seed  
training process
```

Artinya source control code saja belum cukup menjelaskan output model. Makanya ML engineering punya concern tambahan:

- dataset lineage,
- experiment tracking,
- model registry,
- model monitoring,
- retraining policy.

Git vs DVC vs MLflow

Ini confusion yang super common.

Git

Pertanyaan:

"Code version yang mana?"

DVC

Pertanyaan:

"Training snapshot exact yang dipakai apa?"

MLflow

Pertanyaan:

"Run ini parameter, metric, artifact, dan dataset context-nya apa?"

Simplified:

```
Git
→ what code?

DVC
→ what data?

MLflow
→ what experiment result?
```

Ketiganya nyambung.

Airflow masuk di mana?

Bayangin pipeline:

```
download
↓
validate
↓
aggregate
↓
features
↓
train
↓
evaluate
```

Kalau sekali jalan manual, gampang. Kalau setiap hari? Kalau validate gagal? Kalau task kelima butuh output task ketiga? Kalau data belum datang? Mulai terasa kita butuh orchestration. Airflow coordinate workflow. Airflow bukan bikin model lebih pintar.

Docker masuk di mana?

Project works di laptop kalian. Di laptop teman:

```
ModuleNotFoundError
different Python
missing system package
```

Classic. Docker bantu package runtime environment. Intuition-nya:

“Daripada saya kasih list instalasi panjang, saya definisikan environment yang konsisten.”

Docker nggak solve bad model atau bad data. Dia solve environment consistency layer.

CI/CD masuk di mana?

Setiap ada pull request, siapa yang memastikan test, lint, docs, dan Docker build jalan? Kalau jawabannya:

“Semoga contributor ingat.”

itu fragile. GitHub Actions automate repository checks. Perhatikan bedanya:

```
code changed
→ CI/CD

model performance degraded
→ monitoring/retraining
```

Dua trigger yang berbeda.

Monitoring masuk di mana?

Begitu service live, kita butuh feedback loop. Operational feedback:

```
request rate
latency
errors
service health
```

Model feedback:

```
prediction
vs
actual
↓
MAE / RMSE
```

Tanpa feedback, production system seperti nyetir mobil tanpa dashboard. Mobilnya mungkin jalan, tapi kita nggak tahu kondisi mesin atau bensinnya.

MLOps bukan checklist tools

MLOps bukan:

```
Airflow [OK]
Docker [OK]
MLflow [OK]
Grafana [OK]
```

Kalau tools ada tapi training nggak reproducible, model nggak traceable, dan performance nggak dimonitor, tools tersebut cuma dekorasi architecture diagram. Fokus kita adalah lifecycle. Tool cuma implementasi.

Studi kasus - NYC Yellow Taxi

Sebelum ngomong MLOps, kita harus jelas dulu ML problem-nya. Karena MLOps yang bagus nggak bisa menyelamatkan problem formulation yang kabur. Use case kita:

Predict jumlah Yellow Taxi pickups di setiap Manhattan taxi zone untuk satu jam ke depan.

Simple kalimatnya, tapi ada beberapa detail penting.

Unit prediction kita

Model tidak predict “NYC demand” secara global. Model predict per:

```
zone
×
target hour
```

Contoh:

```
Zone 161
Target hour: 2025-01-28 18:00
Prediction: 147 pickups
```

Jadi setiap row model merepresentasikan satu combination:

```
one zone × one hour
```

Kenapa satu jam ahead?

Kenapa nggak next 5 minutes? Kenapa nggak besok? Karena satu jam cukup reasonable buat workshop:

- historical lag masih relevant,
- pattern harian masih strong,
- use case mudah dipahami,
- ground truth datang relatif cepat,
- monitoring/replay enak didemokan.

Satu jam juga bikin konsep online feature cukup jelas. Untuk target 18:00, history terbaru yang boleh dipakai adalah sampai 17:00.

Raw data granularity vs model granularity

NYC TLC raw data:

```
one row
=
one taxi trip
```

Contoh:

pickup time	zone
17:03	161

pickup time	zone
17:10	161
17:44	162

Model kita nggak butuh detail individual trip. Kita aggregate:

timestamp	zone_id	trip_count
17:00	161	2
17:00	162	1

Jadi transform utama pertama:

```
trip events
↓
hourly demand time series
```

Ini penting karena model kita sebenarnya learning temporal demand pattern, bukan characteristic individual passenger.

Input apa yang boleh dipakai?

Target:

```
18:00
```

Boleh:

```
17:00 demand
16:00 demand
yesterday 18:00
last week 18:00
recent rolling average
hour=18
day_of_week
```

Tidak boleh:

```
actual 18:00 demand
future 19:00 demand
rolling window yang include 18:00 actual
```

Kenapa? Karena waktu prediction dibuat, actual 18:00 belum diketahui. Kalau model train pakai informasi future, validation metric bisa terlihat keren tapi production prediction impossible. Itulah leakage.

Business interpretation

Kalau model predict high demand di suatu zone, secara conceptual itu bisa bantu:

- fleet positioning,
- dispatch planning,
- capacity planning,
- demand awareness.

Workshop ini bukan claim bahwa model kita siap dipakai NYC operations. Kita pakai scenario yang realistic enough buat lifecycle learning.

Primary metric: MAE

MAE = Mean Absolute Error. Formula intuition:

```
absolute prediction errors
↓
average
```

Contoh:

```
prediction = 150
actual = 160
error = 10

prediction = 80
actual = 70
error = 10
```

MAE ignore direction, focus magnitude. Kalau:

```
MAE = 10
```

intuition-nya:

On average, prediction meleset sekitar 10 pickups per zone-hour.

Itu gampang dijelaskan ke non-ML audience.

Secondary metric: RMSE

RMSE lebih sensitif ke large error. Misalnya dua model punya MAE mirip, tapi salah satunya kadang miss ekstrem. RMSE bisa lebih tinggi. Makanya kita log both:

```
MAE
→ easy average error interpretation

RMSE
→ more sensitive to big misses
```

Kenapa baseline penting?

Baseline kita:

```
prediction(t)
=
demand(t - 24h)
```

Alias:

"Prediksi jam ini = demand jam yang sama kemarin."

Kelihatannya sederhana. Tapi taxi demand punya daily pattern, jadi baseline ini bisa cukup kuat. Ini bagus. Karena main model harus **earn its complexity**. Kalau complex model kalah dari baseline simple, pertanyaan yang sehat adalah:

"Kenapa kita pakai model lebih kompleks?"

bukan:

“Gimana caranya tetap deploy model kompleks ini?”

Success criteria model

Main model considered useful kalau:

```
model MAE  
<  
baseline MAE
```

Tapi di MLOps context, success nggak berhenti di offline metric. System juga harus:

- reproducible,
- servable,
- observable,
- maintainable.

Jadi ML metric adalah satu dimension dari project quality.

Scope limitation

Kita fokus:

```
Yellow Taxi  
Manhattan  
early 2025  
one-hour horizon  
pickup count
```

Kenapa Manhattan saja? Supaya dataset lebih manageable tapi tetap punya banyak zones dan meaningful spatial variation. Workshop goal bukan maximize NYC coverage. Goal-nya punya use case cukup realistic tanpa bikin participant laptop menangis.

Checkpoint

Coba jawab:

1. Model kita predict apa exactly?
2. Satu row model represent apa?
3. Kenapa actual target hour nggak boleh jadi feature?
4. MAE 10 artinya apa secara intuitif?
5. Kenapa baseline lag_24h bukan baseline asal-asalan?

Step 0 - Setup environment

Hi peers Sebelum download dataset beberapa GB, sebelum buka Airflow, sebelum ngetes Docker, kita pastikan dulu environment dasar sehat. Kenapa ini penting? Karena kalau baseline environment belum jelas, setiap error berikutnya jadi ambiguous. Misalnya training fail. Apakah karena code? Atau dependency belum ke-install? Atau Python version salah? Atau kalian ternyata pakai global environment? Kita eliminate uncertainty dari awal.

Goal step ini

Setelah selesai, kalian harus punya:

- repository berhasil di-clone,
- Python 3.11 tersedia lewat uv,
- core dependency ter-install,
- doctor script green,
- pytest bisa jalan,
- ngerti basic folder structure.

Belum perlu Docker jalan. Belum perlu Airflow. Kita build foundation dulu.

1. Clone repository

```
bash
git clone https://github.com/ahmadnafi30/bcc-workshop-mlops.git
cd bcc-workshop-mlops
```

Pause sebentar

Coba lihat:

```
bash
ls
```

atau di Windows explorer. Kalian akan lihat folder seperti:

```
api
dags
data
docs
scripts
src
tests
```

Jangan dihafal. Cukup ingat:

```
src
→ reusable logic

scripts
→ command manusia

dags
→ workflow Airflow

api
→ HTTP interface

tests
→ safety net
```

Nanti tiap folder akan punya konteks.

2. Check uv

```
bash
uv --version
```

Kalau command not found, install uv dulu sesuai OS. Kenapa kita nggak pakai pip + venv manual? Bisa sebenarnya. Tapi workshop butuh satu workflow setup yang konsisten lintas peserta. uv bikin dependency/project environment lebih predictable.

3. Sync environment

```
bash
uv sync
```

Sebelum run, coba tebak

Command ini kira-kira melakukan apa? Bukan sekadar “install package”. Mental model:

```
read pyproject
↓
resolve dependency
↓
prepare .venv
↓
install project
↓
ready
```

What just happened?

Coba check:

```
bash
uv run python --version
```

Expected:

```
Python 3.11.x
```

Kalau 3.11, good. Kalau bukan, jangan lanjut dulu tanpa ngerti kenapa.

4. Doctor script

```
bash
uv run python scripts/doctor.py
```

Doctor check beberapa requirement dasar. Kalian mungkin lihat:

```
[OK] Python 3.11
[OK] import pandas
[OK] import sklearn
...
```

Docker bisa jadi belum ready. Itu nggak masalah untuk Step 0-6. Doctor sengaja bedakan core requirement dengan tool yang baru dibutuhkan nanti.

5. Run tests

```
bash
uv run pytest
```

Kenapa run tests sekarang, padahal kita belum coding? Karena kita butuh **known-good starting point**. Bayangin kalian langsung edit feature code, lalu test fail. Kalau sebelumnya nggak pernah run test, kita nggak tahu:

“Ini fail karena perubahan saya, atau repo dari awal sudah fail?”

Baseline test state menghilangkan ambiguity.

6. Optional: run docs locally

```
bash
uv sync --group docs
uv run --group docs mkdocs serve --dev-addr 127.0.0.1:8002
```

Open:

```
http://127.0.0.1:8002
```

Kalian bisa keep tab docs ini selama workshop.

7. Explore pyproject.toml

Jangan skip ini. Open:

```
pyproject.toml
```

Cari:

- requires-python,
- dependencies,
- dependency-groups,
- pytest config,
- Ruff config.

Pertanyaan buat diri sendiri:

“Kalau saya add package manual pakai pip tapi nggak update project definition, apa yang terjadi di CI?”

Jawaban: CI fresh machine nggak tahu package personal kalian. Makanya dependency harus jadi project metadata.

8. Mini challenge

Tanpa lihat docs, coba jawab:

A

Kalau mau run script `build_features.py`, command mana lebih konsisten?

```
python scripts/build_features.py
```

atau:

```
uv run python scripts/build_features.py
```

Kenapa?

B

Airflow belum terinstall setelah `uv sync` biasa. Apakah repo broken? No. Airflow ada dependency group sendiri.

Common issues

uv command not found

Install `uv`.

Python version nggak 3.11

Check project Python setup. Jangan paksa global interpreter random.

Import error setelah sync

Coba:

```
bash
uv sync
```

lagi dan lihat error resolution.

pytest fail

Read exact failing test. Jangan langsung delete test.

Checkpoint

Sebelum lanjut, pastikan:

```
bash
uv run python --version
uv run python scripts/doctor.py
uv run pytest
```

semuanya masuk akal. Kalau iya, kita masuk Step 1 dan mulai main dengan data real.

Step 1 - Data dan historical replay

Hi peers, sekarang kita mulai pegang data beneran. Sebelum masuk model, kita perlu memastikan satu hal penting:

“Kita sebenarnya sedang memprediksi dari data seperti apa?”

Karena salah satu mistake paling common di ML adalah terlalu cepat ngomongin model padahal data lifecycle-nya belum jelas. Di step ini kita akan:

1. download official NYC TLC data;
2. prepare replay source;
3. inspect data;
4. prepare initial hourly demand history;
5. ngerti kenapa folder data dipisah;
6. ngerti apa yang nanti direplay oleh Airflow.

Goal

Setelah selesai, kalian harus bisa menjawab:

- raw TLC row artinya apa;
- kenapa kita fokus Manhattan;
- kenapa replay source dibuat compact;
- bedanya source, raw, processed;
- kenapa hourly demand perlu complete grid;
- kenapa Jan 1-26 disiapkan di awal;
- apa role warm-up Jan 1-7;
- bagaimana data ini nanti dipakai model dan monitoring.

Kalau cuma berhasil download file tapi nggak bisa jawab pertanyaan itu, step ini belum benar-benar selesai.

1. Sebelum download, lihat problem-nya dulu

Use case kita:

Predict jumlah pickup taxi per Manhattan zone satu jam ke depan.

Raw TLC data punya:

```
one row
=
one taxi trip
```

Model kita butuh:

```
one row
=
one zone × one hour
```

Artinya data tidak bisa langsung masuk model. Ada transformation lifecycle.

2. Bootstrap official dataset

Setelah step ini:

- official taxi data ter-download,
- zone lookup tersedia,
- replay source terbentuk,
- initial historical demand Jan 1-26 siap,
- kalian ngerti raw granularity vs model granularity.

Sebelum membuka dokumentasi dataset

Raw TLC punya satu row per trip. Coba jawab:

“Kalau target kita jumlah pickup per zone per jam, apakah model perlu satu row per individual trip?”

No. Kita akan aggregate. Tapi raw source tetap penting sebagai starting point.

Jalankan bootstrap

```
bash
uv run python scripts/bootstrap_data.py
```

First run bisa lumayan lama karena monthly TLC Parquet cukup besar. Jangan panik kalau terminal kelihatan sibuk.

3. Apa yang sebenarnya didownload?

Dua hal utama.

Yellow Taxi monthly Parquet

Contoh:

```
yellow_tripdata_2025-01.parquet
```

Taxi Zone Lookup

Mapping:

```
LocationID
→ Borough
→ Zone
```

Tanpa lookup, PULocationID hanya angka. Kita butuh tahu mana zone Manhattan.

4. Kenapa base CloudFront URL bisa AccessDenied?

Kalau kalian coba buka folder base URL dan dapat XML AccessDenied, itu normal. Object storage-nya tidak expose directory listing. Mental model:

```
browse folder
[X]

request exact known object
[OK]
```

Script bootstrap tahu exact monthly object URL. Jadi AccessDenied di prefix bukan berarti dataset private.

5. Lihat folder setelah bootstrap

Expected:

```
data/
├── metadata/
│   └── taxi_zone_lookup.csv
└── source/
    ├── tlc/
    │   └── yellow_tripdata_2025-01.parquet
    └── replay/
        └── yellow_tripdata_2025-01.parquet
```

Question:

“Kenapa TLC source dan replay source dua-duanya disimpan?”

Karena responsibility beda.

```
tlc/
→ original downloaded source

replay/
→ compact workshop-ready historical source
```

6. Kenapa replay source lebih kecil?

Original TLC punya banyak columns. Core use case kita terutama butuh:

```
tpep_pickup_datetime
PULocationID
```

Kita filter Manhattan juga. Jadi replay source lebih kecil dan cepat dibaca ulang. Ini penting karena historical simulation akan sering access data tersebut.

7. Inspect metadata zone lookup

Try:

```
bash
uv run python - <<'PY'
import pandas as pd
df = pd.read_csv('data/metadata/taxi_zone_lookup.csv')
print(df.head())
print(df[df['Borough'] == 'Manhattan'].head())
PY
```

Coba jawab:

- LocationID itu apa?
- Borough dipakai buat apa?
- Zone name useful buat apa?

Model pakai zone_id, tapi human debugging sering terbantu zone name.

8. Optional notebook exploration

Run:

```
bash
uv run jupyter lab
```

Open notebook exploration.

```
notebooks/01-data-exploration.ipynb
```

Jangan treat notebook sebagai mandatory full EDA project. Kita cuma sanity check:

- row count;
- date range;
- missing values;
- pickup zones;
- top pickup zones;
- sample hourly aggregation.

Question:

“Apakah source yang kita download masuk akal sebelum masuk pipeline?”

9. Prepare initial historical demand

Run:

```
bash
uv run python scripts/prepare_historical_demand.py
```

Default range:

```
2025-01-01
through
2025-01-26
```

Output:

```
data/processed/demand/2025-01-01.parquet
...
data/processed/demand/2025-01-26.parquet
```

10. Kenapa Jan 1-26?

Karena kita butuh tiga logical segments.

```
Jan 1-7
→ warm-up history

Jan 8-21
→ training

Jan 22-26
→ validation
```

Lag terpanjang:

```
168h = 7 days
```

Makanya Jan 8 adalah titik pertama yang punya one-week history lengkap.

11. Warm-up bukan wasted data

Jan 1-7 memang tidak jadi target training rows complete. Tapi dia menyediakan context untuk:

```
lag_168h
rolling windows
recent history
```

Analoginya:

Kalian mungkin nggak menilai siswa dari minggu orientasi, tapi aktivitas minggu orientasi bisa jadi context buat minggu berikutnya.

Data yang tidak jadi target row tetap bisa useful sebagai history.

12. Inspect satu processed day

Try:

```
bash
uv run python - <<'PY'
import pandas as pd
df = pd.read_parquet('data/processed/demand/2025-01-26.parquet')
print(df.head())
print(df.shape)
print(df['timestamp'].min(), df['timestamp'].max())
PY
```

Expected columns:

```
timestamp
zone_id
trip_count
```

13. Kenapa processed row count bukan raw row count?

Raw:

```
one trip
→ one row
```

Processed:

```
one zone-hour
→ one row
```

Jadi jutaan trip bisa jadi jauh lebih sedikit hourly rows. Aggregation memang mengubah granularity.

14. Complete grid sanity check

Suppose Manhattan punya Z zones. One day expected:

```
24 × Z rows
```

Kenapa? Setiap zone harus punya semua jam. Termasuk jam tanpa pickup.

15. Zero demand vs missing data

Ini distinction important. No trip di zone tertentu jam 03:00:

```
trip_count = 0
```

Bukan:

```
row missing
```

Missing row bisa berarti data pipeline gagal. Zero berarti observed demand memang nol. Kalau semantics ini salah, lag feature nanti juga salah.

16. Total pickup sanity

Optional:

```
bash
uv run python - <<'PY'
import pandas as pd
df = pd.read_parquet('data/processed/demand/2025-01-26.parquet')
print(df['trip_count'].sum())
PY
```

Compare roughly dengan trip count source Manhattan hari itu kalau ingin. Goal:

processed aggregation tidak tiba-tiba double atau kehilangan massive rows.

17. Kenapa data tidak di-commit ke Git?

Generated Parquet cukup besar dan berubah. Git cocok buat:

- code;
- config;
- docs;
- DAG.

Generated data punya lifecycle lain. Nanti DVC membantu snapshot reproducibility.

18. Predict apa yang akan terjadi di Jan 27

Sekarang kita punya history sampai Jan 26. Production-like period dimulai Jan 27. Kalau kita release Jan 27 nanti:

```
replay monthly source
↓
raw Jan 27
↓
processed Jan 27
↓
features extend
```

Coba visualize ini sebelum Airflow masuk.

19. Manual release Jan 27 - optional preview

Kalau mau lihat:

```
bash
uv run python scripts/simulate_daily_data.py --date 2025-01-27
```

Lalu:

```
bash
uv run python scripts/prepare_daily_demand.py --date 2025-01-27
```

Kita nanti ulang concept yang sama via Airflow. Kenapa manual dulu? Karena kalau Airflow gagal, kalian tahu underlying operation-nya apa.

20. Common issue: file sudah ada

Downloader bisa skip file existing. Ini useful supaya rerun bootstrap nggak download semua dari nol. Tapi kalau kalian curiga local file corrupt, jangan assume skip = valid. Inspect/remove intentionally lalu redownload. Idempotency dan caching harus tetap punya integrity check mindset.

21. Common issue: disk space

Monthly TLC files besar. Workshop presenter sebaiknya pre-download sebelum hari H. Participant juga perlu cukup disk space. Kalau download fail random, cek disk sebelum debugging Python logic terlalu jauh.

22. Common issue: internet venue

Ini alasan presenter checklist nanti menyarankan pre-bootstrap data. Workshop objective bukan menguji Wi-Fi. Source data bisa disiapkan sebelum session supaya waktu lebih banyak buat MLOps.

23. Mini challenge

Coba jawab:

Tiga contoh baris pickup

```
17:03 - Zone 161
17:10 - Zone 161
17:44 - Zone 162
```

Setelah hourly aggregation:

```
17:00 zone 161 → ?
17:00 zone 162 → ?
```

Answer:

```
2
1
```

Simple, tapi ini transformation fundamental seluruh project.

24. Checkpoint

Sebelum Step 2, pastikan:

```
data/metadata/taxi_zone_lookup.csv
[0K]

data/source/replay/...
[0K]

data/processed/demand/2025-01-26.parquet
[0K]
```

Dan coba explain:

1. Raw row artinya apa?
2. Model row nanti artinya apa?
3. Why Manhattan only?
4. Replay source beda apa dengan official source?
5. Processed demand beda apa dengan raw trips?
6. Kenapa zero demand perlu row explicit?
7. Jan 1-7 role-nya apa?
8. Jan 8-21 role-nya apa?
9. Jan 22-26 role-nya apa?
10. Kenapa Git nggak simpan generated Parquet?
11. Manual Jan 27 flow seperti apa?
12. Kalau row count processed tidak $24 \times \text{zones}$, apa yang kalian curigai?

Kalau jawabannya clear, kita siap masuk feature engineering + baseline model.

Step 2 - Feature dan baseline model

Sekarang kita punya hourly demand history. Belum ada MLflow. Belum ada Airflow. Belum ada model registry. Bagus. Kita sengaja belajar ML workflow-nya dulu. Kalau model, split, atau feature engineering salah, MLOps tools hanya akan membuat kesalahan itu lebih otomatis.

Goal

Di akhir step ini kalian harus bisa:

- build feature dataset;
- explain lag/rolling features;
- identify leakage;
- create initial training snapshot;
- train naive baseline + HGB;
- read MAE/RMSE;
- decide apakah main model benar-benar beat baseline;
- ngerti kenapa time split chronological.

1. Build features

Run:

```
bash
uv run python scripts/build_features.py
```

Output:

```
data/features/taxi_demand_features.parquet
```

2. Inspect columns

Try:

```
bash
uv run python - <<'PY'
import pandas as pd
df = pd.read_parquet('data/features/taxi_demand_features.parquet')
print(df.columns.tolist())
print(df.head())
PY
```

Cari:

```

hour
day_of_week
is_weekend
lag_1h
lag_2h
lag_3h
lag_24h
lag_168h
rolling_mean_3h
rolling_mean_6h
rolling_mean_24h
target_trip_count

```

3. Jangan cuma lihat nama column - derive satu row manual

Ambil target:

```

Jan 26 18:00
zone tertentu

```

Coba cari processed demand:

```

16:00
17:00
18:00 actual

```

Target leaked. Correct:

```

15:00
16:00
17:00
16:00
15:00
yesterday 18:00
last week 18:00

```

Lalu compare dengan lag columns. Kalau manual derivation match, feature semantics lebih tangible.

4. Leakage check

Pertanyaan untuk setiap feature:

“Apakah value ini sudah known sebelum target hour terjadi?”

Allowed:

```

target hour calendar info
past demand
past rolling average

```

Not allowed:

```

actual target demand
future demand
rolling window containing target

```

Ini rule sederhana yang harus terus dibawa sampai serving.

5. Kenapa rolling shift dulu?

Target 18:00. rolling_mean_3h harus:

```
15:00
16:00
17:00
```

Kalau include 18:00, model dapat answer sebagian. Validation metric bisa kelihatan amazing. Itu bukan model pintar. Itu leakage.

6. Warm-up rows hilang setelah build features

Karena lag_168h perlu one-week history. Coba inspect minimum timestamp feature dataset. Expected sekitar Jan 8. Ini bukan bug. History Jan 1-7 dipakai sebagai context.

7. Kenapa zone_id category?

ID:

```
100
200
```

tidak berarti zone 200 dua kali lebih besar dari zone 100. Identifier harus diperlakukan sebagai category. Ini modeling semantics, bukan formatting.

8. Create initial training snapshot

Run:

```
bash
uv run python scripts/create_training_snapshot.py --cutoff-date 2025-01-26
```

Output:

```
data/snapshots/training/taxi_demand_2025-01-26.parquet
```

Kenapa snapshot sekarang? Karena kita ingin training input frozen. Step 3 nanti DVC formalize reproducibility-nya.

9. Inspect snapshot range

```
bash
uv run python - <<'PY'
import pandas as pd
df = pd.read_parquet('data/snapshots/training/taxi_demand_2025-01-26.parquet')
print(df['timestamp'].min())
print(df['timestamp'].max())
print(df.shape)
PY
```

Question:

“Apakah Jan 27 boleh masuk?”

No. Initial model cutoff Jan 26.

10. Time split

Training code secara dynamic mengambil last 5 days snapshot sebagai validation. Untuk initial snapshot:

```
Jan 8-21  
→ train  
  
Jan 22-26  
→ validation
```

Kenapa time split? Karena deployment juga selalu:

```
past  
→ learn  
  
future  
→ predict
```

Random split kurang realistic untuk forecasting.

11. Baseline prediction

Naive baseline:

```
prediction  
=  
lag_24h
```

Artinya target jam 18:00 pakai demand kemarin jam 18:00. No fit. No hyperparameter. Simple.

12. Sebelum train main model, predict expectation

Question:

“Kalau taxi demand punya strong daily seasonality, apakah baseline ini bisa cukup kuat?”

Yes. Makanya baseline meaningful. Kalau main model tidak beat baseline, itu possible. Jangan assume ML selalu menang.

13. Run manual training

```
bash  
uv run python scripts/train_model.py
```

Terminal print:

```
naive MAE  
naive RMSE  
model MAE  
model RMSE  
model artifact path  
metrics path
```

Read actual numbers. Docs tidak akan claim exact metric karena result harus berasal dari run kalian.

14. MAE

Suppose hypothetical:

```
MAE = 10
```

Interpretasi:

model rata-rata absolute error sekitar 10 pickups per zone-hour.

Gampang communicate.

15. RMSE

RMSE lebih punish big misses. Suppose:

```
Model A  
errors mostly 8-12
```

```
Model B  
mostly 5  
but some errors 80
```

MAE mungkin tidak terlalu jauh. RMSE Model B bisa spike. Jadi RMSE bantu lihat tail error severity.

16. Model vs baseline decision

Case hypothetical:

```
baseline MAE = 14  
model MAE = 10
```

Main model improves. Case:

```
baseline MAE = 9  
model MAE = 10
```

Main model kalah. Apa yang dilakukan? Healthy process:

1. verify feature correctness;
2. verify split;
3. inspect leakage;
4. inspect data;
5. tune reasonably;
6. accept result.

Jangan manipulate validation supaya model menang.

17. Local artifacts

Manual script save:

```
models/taxi_demand_model.joblib  
models/initial_metrics.json
```

Kenapa useful? Participant bisa lihat pain point manual tracking. Kalau experiment kedua datang, kita mulai punya:

```
model file
metrics file
which one belongs together?
```

MLflow nanti solve ini.

18. Inspect metrics JSON

Open:

```
models/initial_metrics.json
```

Coba compare baseline/model values. Question:

“Kalau besok train lagi, file ini overwrite nggak?”

Potentially yes. Nah, itu salah satu motivation experiment tracking.

19. Why HistGradientBoosting?

Bukan karena objectively best taxi model di dunia. Dipilih karena:

- tabular-friendly;
- CPU-friendly;
- training cukup cepat;
- dependency ringan;
- supports categorical feature;
- easy serving;
- workshop time reasonable.

Model choice dipengaruhi learning objective dan operational constraints.

20. Kenapa bukan LSTM/Chronos core?

Advanced forecasting model bisa dicoba. Tapi kalau model membutuhkan huge download/GPU/complex serving, participant bisa habis waktu di environment issue. Core workshop ingin lifecycle visible. Model sophistication adalah extension.

21. Mini exercise: compare metrics

Suppose:

```
baseline:
MAE 12
RMSE 20

model:
MAE 10
RMSE 25
```

Apakah model clearly better? Not fully obvious. Average absolute error improve, but large misses worse. Ini menunjukkan single metric tidak selalu enough untuk real governance. Workshop gate utama MAE, reviewer tetap inspect RMSE.

22. Run tests

```
bash
uv run pytest
```

Feature tests penting karena leakage bug kadang tidak crash. Code bisa run perfectly sambil semantically wrong. Testing target harus include known failure modes.

23. What if actual model loses?

Workshop tetap valid. MLOps lifecycle tidak require main model selalu superior. Justru experiment tracking + registry membantu manage evidence. Candidate yang jelek bisa tercatat tanpa dipromote. Itu normal science.

24. Checkpoint

Pastikan ada:

```
data/features/taxi_demand_features.parquet
[OK]

data/snapshots/training/taxi_demand_2025-01-26.parquet
[OK]

models/taxi_demand_model.joblib
[OK]

models/initial_metrics.json
[OK]
```

Lalu explain:

1. Lag 1h/24h/168h artinya apa?
2. Kenapa rolling shift dulu?
3. Warm-up rows role-nya apa?
4. zone_id kenapa categorical?
5. Kenapa snapshot cutoff Jan 26?
6. Kenapa split chronological?
7. Baseline lag_24h kenapa meaningful?
8. MAE vs RMSE?
9. Model kalah baseline apakah pipeline gagal?
10. Kenapa manual local artifacts mulai painful setelah banyak experiment?
11. Kenapa model yang lebih advanced tidak otomatis lebih suitable buat workshop?
12. Apa failure mode yang test feature seharusnya catch?

Kalau clear, next kita formalize data reproducibility dengan DVC.

Step 3 - Snapshot dan DVC

Sampai Step 2 kita sudah punya feature dataset dan model yang bisa ditrain. Sekarang coba bayangin satu minggu ke depan. Feature dataset bertambah karena Jan 27, Jan 28, Jan 29 masuk. File yang sama:

```
data/features/taxi_demand_features.parquet
```

content-nya berubah. Lalu kita tanya:

“Model initial yang kita train kemarin sebenarnya pakai rows sampai tanggal berapa?”

Kalau jawabannya cuma:

“Pakai file features itu.”

belum cukup. Kita perlu training input yang **frozen dan identifiable**. Nah, di step ini DVC masuk.

Goal

Setelah selesai kalian harus bisa menjelaskan:

- kenapa moving feature file bukan training identity yang bagus;
- apa itu training snapshot;
- anatomy dvc.yaml;
- cmd, deps, outs;
- apa yang terjadi saat dvc repro;
- kenapa DVC cache bukan Git;
- kenapa SHA fingerprint juga dicatat;
- kenapa DVC dan MLflow punya responsibility beda;
- apa yang berubah kalau dependency berubah.

1. Lihat dulu feature dataset yang sekarang ada

Pastikan:

```
data/features/taxi_demand_features.parquet
```

sudah exist. Optional inspect:

```
bash
uv run python - <<'PY'
import pandas as pd
df = pd.read_parquet('data/features/taxi_demand_features.parquet')
print(df.shape)
print(df['timestamp'].min())
print(df['timestamp'].max())
PY
```

Pertanyaan:

“Kalau besok file ini ditambah Jan 27, apakah nama filenya berubah?”

No. Itu source of ambiguity.

2. Buat snapshot manual dulu

Sebelum DVC, jalankan underlying script:

```
bash
uv run python scripts/create_training_snapshot.py --cutoff-date 2025-01-26
```

Output:

```
data/snapshots/training/taxi_demand_2025-01-26.parquet
```

Terminal juga print:

```
file
rows
zones
sha256
```

Pause. Kenapa nama file punya cutoff date? Karena kita ingin identity yang readable. Kenapa masih butuh SHA kalau nama sudah ada? Karena filename bisa sama tapi content bisa berubah kalau overwrite. Readable identity + content fingerprint lebih kuat.

3. Apa yang dilakukan script snapshot?

Actual flow:

```
read feature dataset
↓
filter rows <= cutoff date
↓
create snapshot
↓
build deterministic snapshot name
↓
write Parquet
↓
describe snapshot
↓
calculate SHA256
```

Important: cutoff date menentukan **data mana yang boleh masuk training context**. Snapshot bukan copy arbitrary. Dia punya temporal boundary.

4. Open dvc.yaml

File:

Actual stage kita sederhana:

```
yaml
stages:
  create_training_snapshot:
    cmd: python scripts/create_training_snapshot.py --cutoff-date 2025-01-26
    deps:
      - data/features/taxi_demand_features.parquet
      - scripts/create_training_snapshot.py
      - src/data_versioning/snapshot.py
    outs:
      - data/snapshots/training/taxi_demand_2025-01-26.parquet
```

Jangan hafal YAML. Baca seperti kalimat.

5. cmd - “Kalau perlu reproduce, jalankan apa?”

```
cmd
→ python scripts/create_training_snapshot.py ...
```

DVC perlu tahu command yang menghasilkan output. Jadi DVC bukan magic yang tahu sendiri cara membuat snapshot. Kita yang declare recipe-nya.

6. deps - “Apa yang bisa mengubah hasil?”

Dependencies:

```
feature dataset
snapshot script
snapshot implementation
```

Kenapa source code masuk deps? Karena output bisa berubah walaupun input data sama kalau logic snapshot berubah. Reproducibility bukan cuma data dependency. Code dependency matters.

7. outs - “Artifact apa yang dihasilkan?”

Output:

```
taxi_demand_2025-01-26.parquet
```

DVC track state artifact ini. Mental model:

```
deps
+
cmd
↓
outs
```

Ini mirip build system.

8. Sebelum dvc repro, predict behavior

Suppose snapshot sudah ada dan dependency belum berubah. Apa yang kalian expect?

```
DVC
→ stage up-to-date
→ tidak perlu regenerate unnecessarily
```

Sekarang run:

```
bash
uv run dvc repro create_training_snapshot
```

Observe output. Jangan cuma lihat command selesai. Cari apakah stage rerun atau dianggap unchanged.

9. dvc status

Run:

```
bash
uv run dvc status
```

Goal:

“Apakah current pipeline output masih sesuai dependency state?”

Kalau feature dataset berubah, DVC bisa report stage out-of-date. Itu jauh lebih useful daripada manusia mengingat:

“Kayaknya kemarin file berubah deh.”

10. Experiment kecil: dependency berubah

Kalau workshop environment aman dan presenter punya backup, bisa demo conceptual change. Misalnya feature dataset rebuilt setelah new date. Sekarang:

```
dep hash changes
↓
DVC detects stage stale
↓
dvc repro reruns snapshot stage
```

Jangan edit snapshot output manual lalu menganggap pipeline valid. Output seharusnya berasal dari declared recipe.

11. DVC cache

DVC punya local cache:

```
.dvc/cache/
```

Kenapa cache? Supaya data content tidak perlu disimpan berulang secara naive. Git menyimpan source + metadata. DVC cache menyimpan heavy data content. Separation:

```
Git
→ project history

DVC
→ data artifact state/cache
```

12. “Kalau cache lokal, teammate lain dapat datanya gimana?”

Good question. Production/team usage biasanya pakai DVC remote. Concept:

```
local DVC cache
↓
dvc push
↓
shared remote storage
↓
teammate dvc pull
```

Remote bisa object storage atau supported backend lain. Workshop tidak setup remote karena credential/cloud setup akan mengalihkan fokus. Tapi architecture path-nya jelas.

13. Kenapa snapshot folder tidak masuk Git normal?

Parquet binary bisa cukup besar. Kalau setiap snapshot di-commit directly ke Git:

```
repo size grows
binary diff poor
clone heavy
```

Git bukan ideal storage buat repeatedly changing large ML data artifact. DVC memberi metadata-driven approach.

14. DVC vs .gitignore

Generated data directory di-ignore Git. Apakah berarti data “nggak versioned”? Not necessarily. Git ignore hanya bilang Git tidak store file binary directly. DVC pipeline metadata/fingerprint bisa tetap represent artifact state. Version control system dan artifact storage dipisahkan.

15. Snapshot immutability mindset

Kalau snapshot:

```
taxi_demand_2025-01-26.parquet
```

sudah dipakai training, healthy behavior:

```
jangan diam-diam edit
```

Kalau data sampai Feb 10:

```
buat snapshot baru
taxi_demand_2025-02-10.parquet
```

Ini menjaga old model lineage.

16. SHA256 - coba pikir kenapa useful

Terminal print SHA. Suppose dua machine punya file:

```
taxi_demand_2025-01-26.parquet
```

Nama sama. SHA sama? Kalau iya, strong evidence content sama. Kalau beda, content beda. MLflow nanti log SHA ini. Jadi:

```
DVC snapshot
↓
SHA fingerprint
↓
MLflow run metadata
```

Cross-tool lineage mulai terbentuk.

17. DVC vs MLflow - jangan jawab “sama-sama versioning”

DVC fokus:

“Bagaimana data artifact ini direproduce dari dependencies?”

MLflow fokus:

“Experiment run ini menggunakan configuration/data/model apa dan hasilnya bagaimana?”

Contoh:

```
DVC:
snapshot X up-to-date?

MLflow:
run Y memakai snapshot X, MAE berapa?
```

Tools-nya complement.

18. DVC vs Airflow

DVC bisa punya pipeline stage. Airflow juga workflow. Apakah redundant? Tidak persis. DVC strong di:

- data/artifact dependency;
- reproducibility;
- content tracking.

Airflow strong di:

- orchestration;
- scheduling;
- task state;
- retries;
- distributed workflow control.

Project kita bahkan punya Airflow training task yang menjalankan:

```
dvc repro
```

Artinya orchestrator memanggil reproducibility tool.

19. Failure scenario

Suppose feature dataset belum ada. Run snapshot script. Expected:

```
FileNotFoundError
→ build_features dulu
```

Bagus. Kenapa? Karena snapshot task punya explicit dependency. Pipeline tidak silently membuat empty snapshot.

20. Mini challenge

Coba jawab apa yang harus berubah di dvc.yaml kalau snapshot cutoff jadi Feb 10. Possible:

```
cmd cutoff
output filename
```

Tapi jangan edit initial stage sembarangan kalau ingin preserve old state. Production design bisa parameterize/multiple stages. Workshop initial DVC stage sengaja static supaya concept mudah. Runtime retraining nanti punya separate snapshot behavior.

21. Checkpoint

1. Moving feature file kenapa bukan identity training yang kuat?
2. Snapshot artinya apa?
3. cutoff-date solve apa?
4. cmd, deps, outs masing-masing?
5. Kenapa code file masuk deps?
6. dvc repro beda apa dengan langsung python script?
7. dvc status jawab pertanyaan apa?
8. Cache dan remote beda apa?
9. Kenapa Parquet snapshot tidak masuk Git biasa?
10. SHA256 membantu lineage bagaimana?
11. DVC dan MLflow beda responsibility?
12. DVC dan Airflow bisa dipakai bareng kenapa?
13. Kenapa old snapshot sebaiknya immutable?

Kalau bisa jawab, DVC sudah lebih dari “Git buat data” di kepala kalian. Next kita pakai snapshot tersebut untuk experiment yang dicatat proper lewat MLflow.

Step 4 - MLflow dan model registry

Sekarang kita punya snapshot yang jelas. Kalau train sekali, manual metrics masih manageable. Tapi mulai run kedua, ketiga, keempat, pertanyaan cepat menumpuk:

“Yang `learning_rate` 0.05 run mana?”

“Run paling bagus pakai snapshot hash apa?”

“Artifact model-nya yang mana?”

“Baseline run ikut tercatat nggak?”

Di sinilah MLflow Tracking masuk. Step ini kita akan **buat experiment, inspect lineage, register candidate, lalu promote champion**.

Goal

Setelah selesai kalian harus bisa menjelaskan:

- Tracking URI;
- Experiment vs Run;
- Parameter vs Metric vs Tag vs Artifact;
- baseline run vs model run;
- dataset fingerprint;
- source run → model version lineage;
- challenger vs champion;
- kenapa promotion explicit;
- kenapa local MLflow state dibagi dengan Docker flow.

1. Start shared MLflow server

Run:

```
bash
uv run python scripts/start_mlflow.py
```

Terminal print:

```
mlflow state: ...
mlflow ui: http://127.0.0.1:5000
```

Open browser:

```
http://127.0.0.1:5000
```

2. Kenapa pakai `scripts/start_mlflow.py`, bukan asal mlflow server?

Helper ini sengaja set:

```
backend store
artifact directory
host/port
allowed hosts
```

dan menyimpan state di:

```
.mlflow/
```

Folder ini nanti di-mount oleh Docker Compose MLflow. Jadi flow:

```
local experiment
↓
register champion
↓
stop local MLflow
↓
docker compose up
↓
same Registry state
```

Tanpa shared state, participant akan bingung karena model seolah hilang saat pindah mode.

3. Tracking URI

Training code perlu tahu MLflow server. Local:

```
http://127.0.0.1:5000
```

Container:

```
http://mlflow:5000
```

Ini same logical service, beda network perspective. Tracking URI adalah configuration, bukan model logic.

4. Sebelum train, buka MLflow UI

Lihat kondisi kosong atau experiment lama. Kalau fresh state, belum ada run. Bagus. Kita akan lihat perubahan setelah command. Sebelum run, predict:

“Berapa run yang akan dibuat?”

Training experiment kita log:

```
naive baseline
+
main model
```

Jadi expect minimal dua run.

5. Run tracked training

```
bash
uv run python scripts/train_with_mlflow.py
```

Actual script reuse:

```
run_mlflow_experiment
```

Stage:

```
initial-validation
```

Terminal output mencakup:

```
tracking uri
experiment name
snapshot
dataset SHA256
baseline MAE + run ID
model MAE + run ID
model URI
```

Jangan langsung copy model run ID saja. Read semua relation-nya.

6. Experiment vs Run

MLflow Experiment = logical grouping. Di dalamnya ada Runs. Analogi:

```
Experiment
→ folder project penelitian

Run
→ satu percobaan
```

Project taxi demand punya experiment logical yang sama, tapi banyak runs dari baseline, candidate, retraining.

7. Buka baseline run

Cari run naive-24h. Inspect:

Params

Model type / dataset context.

Metrics

MAE, RMSE.

Tags

Task/stage. Kenapa baseline ikut MLflow? Karena comparison evidence juga harus traceable. Kalau baseline cuma print terminal lalu hilang, future reviewer kehilangan context.

8. Buka main model run

Cari hist-gradient-boosting. Inspect:

```
Parameters
Metrics
Tags
Artifacts
```

Pertanyaan:

“Apa yang membedakan parameter dan metric?”

Parameter adalah input/config. Metric adalah measured output. Example:

```
learning_rate
→ parameter

mae
→ metric
```

9. Dataset metadata

Cari fields seperti:

```
dataset_snapshot
dataset_sha256
dataset_rows
dataset_zones
```

Sekarang connect ke Step 3. DVC memberi snapshot/fingerprint. MLflow run menyimpan reference context itu. Jadi kita bisa bilang:

“Run ini memakai exact snapshot fingerprint ini.”

Ini lineage nyata.

10. Artifact

Model run punya model artifact. Kenapa bukan cuma joblib arbitrary file? MLflow model packaging bisa menyertakan:

- model artifact;
- flavor metadata;
- signature;
- input example.

Artifact linked ke run. Jadi model file nggak orphan.

11. Input example dan signature

Inspect model metadata kalau tersedia. Signature menjelaskan expected input/output schema. Kenapa useful? Serving nanti perlu compatible input. Signature bukan guarantee semua semantics benar, tapi membantu contract visibility.

12. Compare runs

Pakai MLflow compare UI. Bandingkan baseline dan model. Questions:

- model MAE lebih rendah?
- RMSE bagaimana?
- dataset context sama?
- stage sama?

Comparison UI baru meaningful kalau logging metadata konsisten.

13. Jangan otomatis percaya run terbaru

Newest run bukan necessarily best run. Version number/time bukan quality. Kita harus compare metrics/context. Ini pattern yang nanti sama di Model Registry.

14. Copy model run ID

Terminal atau UI punya model run ID. Run:

```
bash
uv run python scripts/register_model.py --run-id "RUN_ID"
```

Default alias:

```
challenger
```

Output:

```
name
version
alias
uri
```

Sekarang Tracking run berubah status? Bukan. Run tetap run. Registry membuat **model version reference** ke run artifact.

15. Tracking vs Registry secara visual

```
MLflow Tracking
├── Run abc123
│   ├── params
│   ├── metrics
│   └── model artifact
│       └── register
MLflow Model Registry
├── taxi-demand-forecasting-model
│   └── Version 1
│       └── source run abc123
```

Ini relation penting banget.

16. Kenapa alias awal challenger?

Karena new registered model belum automatically production-approved. New != good. Registered != champion. Alias challenger berarti:

"Candidate ini layak direview."

17. Inspect Model Registry

Open registered model. Lihat:

- model name;
- version;
- source run;
- alias;
- artifact link.

Try click source run. Kalian harus bisa traverse:

```

model version
↓
experiment run
↓
metrics
↓
dataset SHA

```

Kalau bisa, lineage berhasil.

18. Promotion

Setelah review, run:

```

bash
uv run python scripts/promote_model.py --version "VERSION"

```

Default target alias:

```

champion

```

Now Registry:

```

champion
→ selected version

```

19. Kenapa promotion bukan rename file?

Tanpa registry, mungkin flow:

```

model_v1.joblib
model_v2.joblib
model_final.joblib
model_production.joblib

```

Mana actual production? Ambiguous. Alias memberi semantic pointer yang explicit dan queryable.

20. Champion dan challenger bisa point ke version yang sama?

Pada initial setup, bisa saja newly registered version punya challenger lalu juga dipromote champion. Alias roles tetap beda secara semantics. Later:

```

champion → v1
challenger → v2

```

lebih typical.

21. Rollback thought experiment

Suppose:

```

champion → v3

```

Setelah deployment-like evaluation ternyata v3 buruk. Old v2 masih registered. Rollback concept:

```
champion
v3 → v2
```

FastAPI tetap query alias champion. Source code tidak perlu edit. Ini benefit alias-driven serving.

22. Apa yang tidak otomatis dilakukan Registry?

Registry tidak otomatis tahu:

- model fair atau tidak;
- business KPI bagus atau tidak;
- inference latency acceptable atau tidak;
- data quality safe atau tidak.

Registry manage lifecycle metadata/version. Decision governance tetap kita design.

23. Run naming dan tags

Good run naming membantu UI readable. Tags membantu filter. Bad:

```
run_123
run_124
```

Better context:

```
hist-gradient-boosting
stage=initial-validation
```

Naming bukan core correctness, tapi operational usability matters.

24. Experiment failure scenario

Suppose baseline run logged, model training fail. MLflow bisa punya partial history. Itu okay. Experiment tracking juga mencatat failure context tergantung implementation. Jangan delete evidence hanya supaya UI terlihat bersih. Failed runs bisa useful buat debugging.

25. MLflow state dan Git

Folder:

```
.mlflow/
```

di-ignore Git. Kenapa? Itu runtime tracking database/artifact state, bukan source. Git track code/config. MLflow track experiment state. Again, different responsibility.

26. Mini challenge

Tanpa lihat docs, trace prediction model production:

```
champion alias
→ ?
→ ?
→ dataset SHA
```

Expected:

```

champion alias
→ model version
→ source MLflow run
→ dataset metadata/SHA

```

Kalau chain ini natural, Model Registry concept sudah masuk.

27. What if model kalah baseline?

Tracked run tetap valuable. Jangan delete. MLflow menunjukkan evidence bahwa attempt tidak improve baseline. Training DAG later bisa decide:

```

model_beats_baseline = false
↓
do not register challenger

```

Tracking dan promotion adalah separate stages.

28. Checkpoint

1. Kenapa start_mlflow helper dibuat?
2. Tracking URI buat apa?
3. Experiment vs Run?
4. Parameter vs Metric?
5. Tag vs Artifact?
6. Kenapa baseline ikut dialog?
7. Dataset SHA connect Step 3 ke MLflow bagaimana?
8. Run ID role-nya apa?
9. Tracking run dan Registry model version relation?
10. Kenapa default alias challenger?
11. Champion alias solve hard-coded version bagaimana?
12. Rollback model concept-nya apa?
13. Kenapa .mlflow tidak masuk Git?
14. Model kalah baseline masih perlu dicatat kenapa?

Kalau semua clear, kalian sudah punya traceability dari data snapshot sampai production model alias. Next kita orchestrate workflow-nya di Airflow.

Kebutuhan format model di versi project saat ini

Project menetapkan `mlflow>=3.16.1,<4.0`. Training melog model sklearn memakai format skops; `src/tracking/mlflow_tracking.py` juga mengizinkan trusted types internal yang diperlukan `HistGradientBoostingRegressor` saat artifact dimuat kembali. Pertahankan versi minimum project saat menyiapkan environment agar model yang diregister dapat di-load oleh serving API.

Step 5 - Airflow orchestration

Nah peers, sekarang timing-nya pas banget buat masuk Airflow. Kenapa baru sekarang? Karena dari Step 1 sampai Step 4 kalian sudah pernah menjalankan proses dasarnya secara manual. Kalian sudah tahu rasanya release data, prepare demand, build features, bikin snapshot, train model, dan track experiment. Itu penting. Kalau Airflow dikenalkan sebelum kalian tahu proses manualnya, yang terlihat cuma kotak-kotak di UI:

```
task A
↓
task B
↓
task C
```

tapi kalian nggak ngerti kotak itu sebenarnya melakukan apa. Sekarang kita punya context.

Cara berpikir setelah step ini

Setelah selesai, kalian bukan cuma bisa click Trigger DAG. Kalian harus bisa menjelaskan:

- kenapa project punya tiga DAG;
- satu task responsibility-nya apa;
- dependency terbentuk dari mana;
- parameter `replay_date` dipakai bagaimana;
- apa yang lewat XCom;
- kenapa DataFrame besar tidak lewat XCom;
- kenapa business logic tetap di src;
- gimana membaca task log;
- gimana berpikir kalau satu task fail;
- apa arti idempotent dalam workflow replay.

Jadi target kita bukan:

"Airflow UI berhasil terbuka."

Target kita:

"Saya ngerti kenapa graph ini bentuknya seperti ini."

Hasil yang perlu terlihat di laptop

Setelah step ini:

- Airflow local jalan,
- tiga DAG visible,
- daily replay dipicu,
- kalian ngerti task state/log,
- ngerti XCom,
- ngerti kenapa DAG dipisah.

1. Install Airflow group

Bagian ini hanya diperlukan untuk mode standalone lokal. Jika kamu memakai Airflow Docker Compose, skip instalasi group ini dan lanjut ke catatan Compose di bagian start. Core environment kita sengaja nggak langsung install Airflow karena dependency-nya cukup berat. Run:

```
bash
uv sync --group airflow
```

First install bisa agak lama. Itu normal. Sambil menunggu, coba ingat lagi perbedaan:

```
uv sync
→ core project + dev environment

uv sync --group airflow
→ core project + Airflow dependencies
```

Kenapa dipisah? Karena tidak semua task di repo butuh Airflow. Participant yang cuma mau run feature engineering nggak perlu bayar installation cost Airflow dari awal.

2. Start Airflow lokal (standalone)

Panduan langkah ini memakai Airflow lokal. Start:

```
bash
uv run --group airflow python scripts/start_airflow.py
```

Buka <http://localhost:8080>. Password awal muncul di terminal saat pertama kali Airflow dijalankan. Jika terlewat, cek file `.airflow/simple_auth_manager_passwords.json.generated` di root repo.

Catatan: Kalau workshop-mu memakai Docker Compose

Jangan start standalone lokal bersamaan. Compose dan standalone menyimpan metadata serta run history di tempat berbeda, sehingga UI-nya bisa tampak seperti dua Airflow yang berbeda. Start service Compose lalu cek port host:

```
bash
docker compose up -d --build airflow
docker compose ps airflow
```

Buka port di sisi kiri pemetaan PORTS. Jika output menunjukkan `127.0.0.1:18081->8080/tcp`, gunakan <http://localhost:18081>. Untuk melihat credential Compose, jalankan `docker compose exec airflow cat /airflow/simple_auth_manager_passwords.json.generated`. Jika ingin menjalankan DAG `taxi_initial_training` di container, aktifkan mode DVC tanpa SCM terlebih dahulu karena image tidak menyertakan folder `.git`:

```
bash
docker compose exec airflow dvc config core.no_scm true --local
```

Ulangi konfigurasi ini jika container dibuat ulang. Konfigurasi hanya berlaku di container; latihan versioning DVC pada host tetap memakai Git checkout. Jangan langsung click semua menu. Pause sebentar. Pertanyaan pertama:

“Airflow sekarang sebenarnya tahu DAG kita dari mana?”

Jawabannya: dari DAG files yang ada di folder project. Project kita punya tiga:

```
taxi_daily_replay
taxi_initial_training
taxi_model_monitoring
```

Kalau salah satu nggak muncul, jangan lanjut seolah normal. Itu berarti ada kemungkinan:

- DAG parsing error;
- import error;
- Airflow environment belum benar;
- file tidak terbaca.

3. Kenapa ada tiga DAG, bukan satu DAG super panjang?

Coba bayangin satu mega DAG:

```
new data
↓
aggregate
↓
features
↓
train
↓
register
↓
monitor
↓
retrain
↓
promote
```

Kelihatannya lengkap. Tapi logic trigger-nya aneh. Apakah setiap data baru harus langsung train model? Tidak. Apakah setiap monitoring run harus release new data? Juga tidak. Jadi kita pisah berdasarkan lifecycle responsibility.

taxi_daily_replay

Trigger concept:

“Ada satu logical production day baru yang mau direlease.”

taxi_initial_training

Trigger concept:

“Kita ingin membuat initial candidate model dari reproducible snapshot.”

taxi_model_monitoring

Trigger concept:

“Kita ingin evaluate champion dan decide apakah retraining perlu.”

Ini desain yang jauh lebih sehat daripada satu DAG raksasa.

4. Open daily replay Graph

Masuk Graph view. Expected flow:

```

get_replay_date
  ↓
release_batch
  ↓
validate_batch
  ↓
aggregate_demand
  ↓
rebuild_features

```

Sekarang coba mapping dengan manual flow yang sebelumnya kalian jalankan.

```

simulate_daily_data.py
→ release_batch

validation
→ validate_batch

prepare_daily_demand.py
→ aggregate_demand

build_features.py
→ rebuild_features

```

Nah, ini moment penting:

Airflow tidak menciptakan data logic baru. Airflow meng-orchestrate logic yang sudah kita punya.

5. Buka source DAG sambil lihat Graph

File:

```
dags/taxi_daily_replay.py
```

Kalian akan lihat decorator:

```

python
@dag(
    dag_id="taxi_daily_replay",
    schedule=None,
    ...
)

```

Apa arti schedule=None? Workshop kita pakai manual trigger. Kenapa? Karena logical time kita historical. Kita mau bisa:

```

09:00 real-world time
→ replay 2025-01-27

09:10 real-world time
→ replay 2025-01-28

```

Kalau pakai schedule real harian, kita harus nunggu besok buat demo tanggal berikutnya. Historical replay mempercepat clock, bukan mengubah dependency logic.

6. Param replay_date

DAG punya param:

```
replay_date
```

Default:

```
2025-01-27
```

Kenapa parameter lebih bagus daripada edit source code setiap demo? Bad:

```
ubah Python file ke Jan 27
run
ubah file ke Jan 28
run
```

Sekarang Git diff penuh perubahan yang sebenarnya cuma runtime input. Better:

```
same DAG definition
+
different run parameter
```

Parameter memisahkan workflow definition dari runtime value.

7. Task get_replay_date

Actual logic membaca Airflow context:

```
context
↓
params
↓
replay_date
```

Output task ini string tanggal. Kecil banget. Kenapa sampai dibuat task sendiri? Karena dia membuat runtime input visible dalam graph dan bisa menjadi upstream dependency untuk task berikutnya.

8. Task release_batch

Task ini call reusable function:

```
release_replay_batch
```

Actual logic ada di:

```
src/orchestration/daily_pipeline.py
```

Function menentukan monthly source berdasarkan date. Misalnya:

```
replay_date = 2025-01-27
↓
month = 2025-01
↓
source = yellow_tripdata_2025-01.parquet
↓
output = data/raw/trips/2025-01-27.parquet
```

Jadi DAG tidak punya Pandas logic untuk filter daily rows. Dia delegate.

9. Sebelum trigger: predict dulu apa yang akan terjadi

Kita trigger:

```
replay_date = 2025-01-27
```

Sebelum click Trigger, coba jawab:

1. File raw apa yang akan muncul?
2. File processed apa yang akan muncul?
3. Feature dataset akan berhenti sampai tanggal mana?
4. Kalau release_batch fail, apakah aggregate_demand boleh jalan?

Expected reasoning:

```
raw:
data/raw/trips/2025-01-27.parquet

processed:
data/processed/demand/2025-01-27.parquet

features:
rebuilt through Jan 27

upstream fail:
downstream blocked
```

Baru trigger. Learning lebih meaningful kalau kalian punya hypothesis sebelum lihat result.

10. Observe task states

Saat DAG run, task bisa berada di state seperti:

```
scheduled
queued
running
success
failed
skipped
```

Warna di UI bukan dekorasi. State memberi observability ke workflow. Kalau manual Bash sequence, beberapa jam kemudian kalian mungkin lupa command mana fail. Airflow persist run/task state.

11. Buka release_batch logs

Click task. Open log. Jangan cuma cari tulisan success. Cari:

- logical date;
- output path;
- row count;
- exception kalau ada.

Kalau output bilang file Jan 27 dibuat, check juga filesystem. Green task state dan actual artifact sebaiknya correlate.

12. validate_batch dan fail-fast thinking

Task berikutnya validate raw batch. Kenapa nggak langsung aggregate? Bayangin source batch ternyata:

- kosong;
- tanggal salah;
- schema missing;
- timestamp invalid.

Kalau langsung transform, problem bisa menyebar lebih jauh. Flow sehat:

```
ingest
↓
validate
↓
transform
```

Fail early memberi error lebih dekat ke root cause.

13. Detail actual code: dependency metadata yang nggak dibaca

Di DAG kalian akan lihat validate_batch menerima release_info. Di dalam function ada assignment ke underscore karena content release_info memang tidak dibutuhkan validation logic. Pertanyaan natural:

“Kalau isi metadata-nya nggak dipakai, kenapa tetap dikirim?”

Karena object tersebut membentuk task dependency.

```
release_batch
↓ output reference
validate_batch
```

Validation bisa menemukan file secara deterministic dari replay_date, tapi dependency tetap explicit. Jadi output di sini punya dua fungsi potensial:

```
data value
atau
dependency signal
```

Ini subtle, tapi bagus banget buat memahami orchestration.

14. XCom - apa yang sebenarnya berpindah?

Task return dictionary kecil. Contoh aggregation metadata:

```
date
rows
zones
pickups
path
```

Ini cocok buat XCom. Sekarang bayangin return full taxi DataFrame jutaan rows. Airflow harus serialize dan menyimpan data besar lewat metadata backend. Bad idea. Mental model:

```
XCom
→ sticky note

Parquet
→ actual package
```

Sticky note bilang:

“Paket ada di path ini, rows sekian.”

Sticky note bukan tempat menaruh isi seluruh gudang.

15. aggregate_demand

Task membaca daily trip parquet + zone lookup lalu reuse aggregate_hourly_demand. Output complete hourly demand. Airflow nggak perlu ngerti detail complete zone-hour grid. Itu domain logic preprocessing. Again:

```
Airflow
→ orchestration

preprocessing module
→ transformation semantics
```

16. rebuild_features

Setelah daily demand baru tersedia, feature dataset dibangun ulang dari history start sampai replay_date. Kenapa nggak cuma build feature hari baru? Karena lag/rolling punya history dependency. Untuk workshop, rebuild relevant history lebih simple dan reliable daripada incremental feature update. Production scale besar mungkin optimize incremental compute. Correctness dulu, optimization nanti.

17. Trigger Jan 28

Setelah Jan 27 green, trigger Jan 28. Sebelum run, predict:

```
raw Jan 28
processed Jan 28
feature dataset extends to Jan 28
```

DAG sama. Parameter beda. Ini reusable workflow definition.

18. Idempotency experiment

Apa yang terjadi kalau Jan 28 di-trigger lagi? Workflow sebaiknya menghasilkan state yang predictable, bukan duplicate random data. Retry itu normal. Network transient fail, worker restart, atau rerun manual bisa terjadi. Idempotency principle:

Rerun logical operation yang sama harus aman dan predictable.

19. Buka taxi_initial_training

Graph:

```
create_snapshot
↓
train_model
↓
register_candidate
```

create_snapshot menjalankan DVC repro. Artinya training tidak asal baca moving latest file. train_model log baseline + candidate ke MLflow. register_candidate punya gate:

```
model beat baseline?
├─ no → not_registered
└─ yes → register_challenger
```

Jadi Airflow meng-orchestrate model lifecycle policy, bukan sekadar train.py.

20. Kenapa register_candidate task terpisah?

Kalau training success tapi Registry unreachable, failure-nya beda dari model training failure. Task boundary membuat incident lebih observable:

```
train_model
[OK]

register_candidate
[X]
```

Kita jadi tahu layer mana yang perlu di-debug.

21. Buka taxi_model_monitoring

Graph:

```
evaluate_model
↓
maybe_retrain
```

Parameter:

```
degradation_multiplier
min_samples
recent_limit
```

Kenapa configurable? Supaya monitoring policy bisa dieksperimenkan tanpa edit source. Dalam production, policy change tetap sebaiknya governed.

22. No-op adalah valid automation result

Kalau result:

```
retrain_recommended = false
```

maybe_retrain return not_needed. DAG success. Apakah automation gagal karena nggak bikin model baru? No. Decision itu sendiri adalah work. Automation yang selalu harus menghasilkan action justru suspicious.

23. Airflow vs script - value tambahannya apa?

Manual script sudah bisa melakukan operation. Airflow menambah:

- dependency graph;
- task state;
- run history;
- logs;
- parameterized runs;
- retry/scheduling capability;
- UI visibility.

Jadi Airflow bukan pengganti Python. Airflow meng-orchestrate Python work.

24. Controlled failure exercise

Kalau workshop time cukup, lakukan safe failure. Misalnya trigger logical date yang source-nya belum siap di local copy. Observe:

1. Task mana fail?
2. Downstream apa yang tidak jalan?
3. Log bilang apa?
4. Setelah dependency fixed, apakah perlu ulang semua?

Failure exercise sering lebih mendidik daripada hanya happy path. Pastikan state dipulihkan setelah demo.

25. Debugging checklist

Kalau DAG nggak muncul:

```
cek import
cek parse error
cek Airflow dependency
```

Kalau DAG muncul tapi task fail:

```
which run?
which task?
what exception?
input path ada?
upstream result?
dependency service reachable?
```

Kalau task green tapi output salah:

```
inspect generated artifact
inspect reusable domain function
run function/script manually
```

Problem task tidak selalu berarti problem Airflow.

Mini discussion

Scenario A

Feature engineering logic salah. Fix di src/features, bukan copy fix ke DAG.

Scenario B

aggregate_demand harus menunggu validation. Concern-nya dependency DAG.

Scenario C

Daily pipeline mau dijadwalkan jam 02:00. Concern-nya Airflow schedule.

Scenario D

Prediction endpoint berubah. Concern-nya FastAPI, bukan Airflow. Boundary thinking ini inti architecture.

Checkpoint

Coba explain tanpa lihat docs:

1. DAG itu apa?
2. Task itu apa?
3. schedule=None kenapa dipilih?
4. replay_date solve problem apa?
5. Kenapa upstream metadata bisa dipakai hanya sebagai dependency signal?
6. XCom cocok untuk data seperti apa?
7. Kenapa DataFrame besar tetap di Parquet?
8. Kenapa daily replay dan training dipisah?
9. Kenapa registration task terpisah?
10. No-op monitoring run bisa successful kenapa?
11. Apa itu idempotency?
12. Kalau task merah, debugging mulai dari mana?

Kalau bisa jawab dengan reasoning, Airflow section ini sukses. Next kita pindah ke model serving: gimana champion akhirnya bisa diminta prediction lewat HTTP?

Step 6 - Serving melalui FastAPI

Sekarang kita punya champion model di MLflow Registry. Kalau model cuma hidup di Registry, application lain belum bisa pakai. Kita butuh serving interface. Di step ini kita akan start FastAPI, inspect endpoint, kirim prediction, dan trace apa yang terjadi dari request sampai model response.

Goal

Setelah selesai kalian harus bisa:

- explain health vs model readiness;
- inspect /model-info;
- send /predict;
- understand 422 vs 503;
- understand online feature construction;
- understand champion alias resolution;
- verify prediction logging;
- inspect /metrics;
- explain dependency injection/testing concept.

1. Pastikan MLflow jalan

Kalau local server sudah stop:

```
bash
uv run python scripts/start_mlflow.py
```

Open:

```
http://127.0.0.1:5000
```

Check champion alias.

2. Pastikan champion alias ada

Model Registry harus punya:

```
champion
→ version X
```

Kalau belum:

```
bash
uv run python scripts/promote_model.py --version "VERSION"
```

Question:

“Kalau champion nggak ada, apakah FastAPI tahu model mana yang approved?”

No.

3. Pastikan historical demand cukup

Untuk target Jan 28 18:00, online feature provider perlu up to 168h history. Jadi processed demand previous days harus exist. Check:

```
data/processed/demand/
```

Kalau history incomplete, prediction seharusnya fail clearly.

4. Start FastAPI

New terminal:

```
bash
uv run uvicorn api.main:app --reload
```

Open:

```
http://127.0.0.1:8000/docs
```

Ini Swagger UI.

5. Sebelum click, lihat endpoint list

Expected:

```
GET /health
GET /model-info
POST /predict
```

Operational hidden endpoint:

```
GET /metrics
```

Why hidden? Karena /metrics bukan business API participant/client utama.

6. GET /health

Swagger atau curl:

```
bash
curl http://127.0.0.1:8000/health
```

Expected:

```
json
{"status": "ok"}
```

Interpretasi:

FastAPI process alive.

Bukan:

model definitely ready.

7. Kenapa health tidak load champion?

Kalau /health tergantung MLflow:

```
MLflow down
↓
health red
```

Kita tidak tahu apakah API process mati atau dependency down. Simple health signal membantu isolate layer. Readiness probe production bisa dibuat lebih comprehensive separately.

8. Test /model-info

```
bash
curl http://127.0.0.1:8000/model-info
```

Response include:

```
model_name
model_version
model_alias
run_id
model_uri
```

Coba compare model_version dengan Registry UI. Harus match champion alias.

9. Prediction request - sebelum send, predict internal flow

Request:

```
json
{
  "zone_id": 161,
  "target_datetime": "2025-01-28T18:00:00"
}
```

Sebelum Try it out, coba sebutkan internal steps. Expected:

```
validate schema
↓
build online features from history
↓
resolve champion
↓
load/reuse model
↓
predict
↓
clip negative if needed
↓
log prediction
↓
observe Prometheus metric
↓
return response
```

Baru send.

10. Inspect response

Jangan cuma lihat:

```
predicted_trip_count
```

Lihat juga:

```
model_version
model_alias
run_id
model_uri
```

Kenapa? Prediction lineage. Kalau output aneh, kita tahu model mana yang produce.

11. Kenapa client tidak kirim lag feature?

Kalau request harus kirim:

```
lag_1h
lag_24h
rolling_mean_3h
...
```

client harus ngerti feature engineering. Bad coupling. Dengan:

```
zone_id
target_datetime
```

serving layer menjaga feature semantics.

12. Test invalid target time

Try:

```
json
{
  "zone_id": 161,
  "target_datetime": "2025-01-28T18:30:00"
}
```

Expected validation failure / 422. Kenapa? Model contract hourly. 18:30 bukan exact target hour.

13. Apa arti 422?

Roughly:

Request structure diterima, tapi input tidak valid untuk domain/schema prediction.

Contoh:

- invalid zone;
- target minute bukan 00;
- historical feature unavailable.

Bukan server crash.

14. Simulate champion missing conceptually

Kalau champion alias tidak ada, /model-info atau /predict bisa return 503. Apa arti 503?

Service process alive, dependency model temporarily unavailable/not ready.

Ini beda dari 422.

15. 422 vs 503 thought exercise

Request target 18:30

```
422
```

Client/input problem.

MLflow unreachable

```
503
```

Service dependency problem. Different action.

16. Model loader cache

First request mungkin load champion model. Next requests reuse cache. Kenapa? Model load lebih mahal daripada predict. Loader periodically refresh Registry metadata. Jadi:

```
cache fast
+
still detects champion update
```

Tradeoff freshness vs overhead.

17. Prediction log

Setelah successful request, inspect:

```
data/monitoring/predictions.jsonl
```

Optional:

```
bash
tail -n 5 data/monitoring/predictions.jsonl
```

Cari request kalian. Fields:

```
target_datetime
zone_id
predicted_trip_count
model_version
run_id
logged_at
```

Prediction event sekarang siap untuk delayed evaluation nanti.

18. Kenapa log setelah successful predict?

Kalau request invalid dan tidak ada prediction, jangan tulis fake prediction event. Monitoring hanya evaluate actual inference yang terjadi. Event semantics harus clear.

19. Negative output domain constraint

Regression theoretically bisa predict negative. Serving result clip minimum zero. Question:

“Kenapa bukan model training saja yang guarantee?”

Model mathematical output space continuous. Domain knows count cannot negative. Post-processing boundary enforce valid business output.

20. Open /metrics

```
http://127.0.0.1:8000/metrics
```

Cari:

```
taxi_api_requests_total  
taxi_predictions_total  
taxi_model_version_info
```

Sekarang kalian sudah mulai melihat connection FastAPI → Prometheus.

21. Request middleware

Setiap HTTP request lewat middleware. Concept:

```
start timer  
↓  
endpoint  
↓  
response  
↓  
record duration  
↓  
increment request counter
```

Kalau endpoint fail pun status code metric tetap useful.

22. Test multiple requests

Send beberapa prediction. Lihat /metrics lagi. Counter harus naik. Ini good prelude sebelum Step 9 monitoring.

23. Swagger vs curl

Swagger good buat learning. curl good buat automation/simple CLI. API contract sama. Tool client beda. Jangan confuse Swagger sebagai “FastAPI backend”. Swagger hanya interactive docs UI.

24. API testing architecture

Project tests override predictor dependency. Kenapa? Supaya test endpoint tidak butuh real:

- MLflow;
- model;
- Parquet history.

Unit/API contract test fokus satu layer. Integration flow tetap dites lewat workshop stack.

25. Common error: /health works, /predict fails

Jangan bilang API broken. Interpretation:

```
HTTP process
[OK]

prediction dependency/path
[X]
```

Check:

- champion;
- history;
- target;
- MLflow;
- model loading.

Specific diagnosis.

26. Common error: model info version unexpected

Check Registry alias. Maybe challenger baru registered tapi champion belum moved.

Remember:

```
challenger
≠
champion
```

API serve champion only.

27. Common error: history missing

Online feature provider membaca processed daily demand files. Kalau lag_168h source date missing, request fail. Correct. Jangan silently fill zero, karena zero means actual no demand, bukan missing history.

28. Checkpoint

1. /health menjawab apa?
2. /model-info menjawab apa?
3. /predict internal flow?
4. Kenapa client tidak kirim lag features?
5. 422 vs 503?
6. Kenapa champion alias digunakan?
7. Kenapa model di-cache?
8. Prediction log dipakai kapan?
9. Negative prediction diproses bagaimana?
10. /metrics dipakai siapa?
11. Middleware mencatat apa?
12. Swagger role-nya apa?
13. /health green tapi /predict fail artinya apa?
14. Missing history kenapa tidak di-fill zero?

Kalau clear, next kita package seluruh stack pakai Docker dan Compose.

Step 7 - Docker dan Compose

Sebelum step ini kalian mungkin punya beberapa terminal:

```
MLflow
FastAPI
Airflow
```

Kalau tambah Prometheus/Grafana manual, makin ramai. Sekarang kita package runtime dan jalankan stack via Docker Compose. Goal bukan sekadar berhasil docker compose up. Kita ingin ngerti image, container, network, volume, health, dan state continuity.

Goal

Setelah selesai kalian bisa explain:

- image vs container;
- Dockerfile target;
- build vs run;
- host port vs container port;
- localhost trap;
- bind mount vs volume;
- shared .mlflow state;
- healthcheck;
- Compose service dependency;
- logs/ps/down lifecycle.

1. Check Docker

```
bash
docker --version
docker compose version
```

Kalau command missing, bereskan Docker dulu. Windows user biasanya lebih nyaman lewat WSL2 + Docker integration.

2. Stop local processes dulu

Kalau local Uvicorn masih pakai 8000, MLflow 5000, Airflow 8080: Compose akan conflict port. Stop dengan Ctrl+C. Question:

“Apakah stop local MLflow menghapus Registry?”

No. State ada di .mlflow folder.

3. Inspect Dockerfile

Open:

```
docker/Dockerfile
```

Cari targets:

```
base
api
mlflow
airflow
```

Kenapa satu Dockerfile banyak target? Shared base environment, different service runtime.

4. Build one target manually - optional

```
bash
docker build -f docker/Dockerfile --target api -t bcc-mlops-api .
```

Ini membantu understand:

```
Dockerfile
↓
build
↓
image
```

Check:

```
bash
docker images | grep bcc
```

5. Image vs container

Image:

built template.

Container:

running instance.

Kalau build selesai tapi belum run, service belum hidup. Kalau container stop, image masih ada. Lifecycle berbeda.

6. Start full stack

```
bash
docker compose up -d --build
```

First run bisa lama. Airflow dependencies heavy. Jangan interrupt hanya karena beberapa menit.

7. docker compose ps

```
bash
docker compose ps
```

Lihat:

- service;
- status;
- health;
- ports.

Expected services:

```
mlflow
api
airflow
prometheus
grafana
```

Workspace tidak long-running by default.

8. Open all service URLs

Service	URL
MLflow	http://localhost:5000
Airflow	host port pada docker compose ps airflow
FastAPI	http://localhost:8000
Swagger	http://localhost:8000/docs
Prometheus	http://localhost:9090
Grafana	http://localhost:3000

Untuk Airflow, jalankan `docker compose ps airflow` lalu buka host port di sisi kiri pemetaan. Contoh `127.0.0.1:18081->8080/tcp` berarti alamat browser-nya `http://localhost:18081`. Jangan langsung assume all ready kalau browser error pertama detik startup. Healthchecks/startup bisa perlu waktu.

9. Verify MLflow state continuity

Open MLflow. Champion yang dibuat di Step 4 harus tetap ada. Kalau ada, shared state works. Kenapa? Local helper dan Docker service sama-sama use:

```
.mlflow/
```

10. Kalau champion hilang, apa yang dicek?

Jangan retrain dulu. Check:

- local MLflow dulu pakai `start_mlflow.py`?
- `.mlflow` folder ada?
- Compose mount benar?
- current working directory benar?
- permission?

Likely state-path issue, bukan model issue.

11. Test FastAPI inside Compose

```
bash
curl http://localhost:8000/health
curl http://localhost:8000/model-info
```

Kalau health works tapi model-info fail, same dependency reasoning berlaku.

12. Host URL vs service URL

Browser:

```
localhost:5000
```

API container:

```
http://mlflow:5000
```

Kenapa bukan localhost? Karena localhost di API container adalah API container sendiri. Compose DNS resolve service name.

13. Port mapping thought experiment

Compose:

```
8000:8000
```

Kiri host. Kanan container. Kalau change host to:

```
9000:8000
```

browser access:

```
localhost:9000
```

container internal app tetap listen 8000. This distinction useful.

14. Shared data bind mount

Compose mount project data folder. Kenapa? Airflow writes:

```
data/processed/...
```

FastAPI reads same history. Host participant juga inspect same files. One simple shared filesystem for local workshop.

15. Named volumes

Prometheus/Grafana/Airflow punya persistent service state. Named volume survive container recreation. Container disposable. Data state can persist.

16. Check logs

```
bash
docker compose logs -f api
```

Try MLflow: Lihat logs. Stop follow dengan Ctrl+C. Container tetap running.

17. Logs bukan same as metrics

Logs:

```
detailed event/error
```

Metrics:

```
aggregated numeric trend
```

Nanti dua-duanya useful. Jangan berharap Grafana menggantikan stack traces.

18. Healthcheck

Container status bisa:

```
running
```

tapi health:

```
starting
```

Application belum ready. Compose depends_on health behavior membantu startup ordering.

19. Workspace one-shot container

Run:

```
bash
docker compose run --rm workspace python scripts/doctor.py
```

Ini start temporary workspace container. Command selesai → container removed. Why useful? Kita bisa run project tool dalam same containerized environment tanpa install local extra tool.

20. Container inspect vs source changes

Kalau code copied into image lalu source host berubah, running container tidak otomatis update unless bind-mounted source or rebuild. Project production-like services use built image behavior. So after source change:

```
bash
docker compose up -d --build
```

may be needed.

21. docker compose down

```
bash
docker compose down
```

Stops/removes containers + network. Named volumes remain.

22. docker compose down -v

```
bash
docker compose down -v
```

Also removes named volumes. Use carefully. Prometheus/Grafana/Airflow state reset. Shared .mlflow bind folder remains because it is host path.

23. Full reset vs normal restart

Normal:

```
bash
docker compose down
docker compose up -d
```

keeps volumes. Full reset:

```
bash
docker compose down -v
```

destructive. Jangan gunakan full reset sebagai first debugging instinct. Bisa menghapus evidence/state yang sebenarnya useful.

24. Scenario: API cannot connect MLflow

Error:

```
connection refused localhost:5000
```

Inside API container, likely wrong hostname. Correct internal:

```
mlflow:5000
```

This is classic container networking bug.

25. Scenario: port already allocated

Compose error host port 8000 used. Check local process. Linux:

```
bash
ss -ltnp | grep 8000
```

or stop previous Uvicorn. Problem host binding, bukan Dockerfile necessarily.

26. Scenario: Prometheus starts before API healthy?

depends_on + healthcheck design helps. Still, always inspect target page after stack ready. Distributed services have startup timing.

27. Why not install everything one container?

Because service lifecycle different. If Grafana restart, API shouldn't necessarily restart. If API scale 3 replicas, MLflow need not scale same. Separation creates flexibility.

28. Docker image as delivery artifact

Step 8 GitHub Actions later build same service targets and push to GHCR. So local Docker learning directly connects CI/CD.

```
Dockerfile
↓
local build
↓
CI build
↓
registry image
```

One artifact definition across lifecycle.

29. Checkpoint

1. Image vs container?
2. Build vs run?
3. Kenapa local process harus stop sebelum Compose?
4. Host localhost vs container service name?
5. Apa arti 9000:8000?
6. Bind mount vs named volume?
7. Kenapa .mlflow shared?
8. Running vs healthy?
9. Workspace service buat apa?
10. Source change kapan perlu rebuild?
11. down vs down -v?
12. Port conflict debug di layer mana?
13. Kenapa service dipisah container?
14. Docker image connect ke CI/CD bagaimana?

Kalau clear, next kita lihat bagaimana GitHub otomatis verify dan publish image tersebut.

Step 8 - CI/CD dan container delivery

Sekarang project sudah punya code, docs, API, Docker, dan multi-service stack. Kalau repo dikerjakan tim, pertanyaannya berubah:

“Gimana caranya kita tahu satu perubahan aman sebelum masuk stable branch?”

Kita nggak mau quality bergantung pada contributor ingat checklist manual. Di step ini kita baca GitHub Actions workflow **sebagai system**, bukan sebagai YAML yang harus dihafal.

Goal

Setelah selesai kalian harus bisa menjelaskan:

- trigger workflow;
- workflow, job, step;
- branch policy;
- quality job;
- matrix Docker build;
- cache;
- concurrency;
- permissions;
- develop → main flow;
- Continuous Delivery ke GHCR;
- kenapa tested commit harus sama dengan published commit.

1. Lihat branch flow dulu

Repository policy:

```
feat/* / fix/* / docs/* / chore/*  
  ↓  
develop  
  ↓  
main
```

Kenapa workflow ini penting sebelum baca CI? Karena CI punya branch-policy job yang encode flow tersebut. CI bukan hanya test code. Dia juga verify contribution process.

2. Open ci.yml

File:

```
.github/workflows/ci.yml
```

Jangan baca semua syntax. Mulai atas.

```
name  
on  
permissions  
concurrency  
jobs
```

Ini skeleton GitHub Actions workflow.

3. name

```
CI
```

Simple. Nama muncul di Actions UI dan workflow_run trigger CD. Naming jelas membantu chaining automation.

4. on - trigger

CI jalan pada:

```
push
pull_request
workflow_dispatch
```

Push branches include main, develop, dan branch pattern work. Pull request focus main/develop. workflow_dispatch berarti manual run tersedia. Question:

“Kenapa feature branch push juga run CI?”

Supaya developer dapat feedback sebelum PR.

5. permissions

CI:

```
contents: read
```

Kenapa read only? Quality job tidak perlu write repo/package. Least privilege. Kalau workflow tidak butuh write, jangan kasih write.

6. concurrency

Config:

```
group: ci-...
cancel-in-progress: true
```

Scenario:

```
push commit A
↓
CI A running

push commit B 30 sec later
↓
CI B starts
```

CI A sudah obsolete untuk latest branch state. Cancel old run menghemat runner.

7. Job 1 - Branch policy

Job hanya pada pull_request. Dia baca:

```
BASE_BRANCH
HEAD_BRANCH
```

Rules:

```
to main
→ head must be develop

to develop
→ head must be feat/*, fix/*, docs/*, chore/*
```

Coba scenario.

feat/new-api → develop

[OK]

develop → main

[OK]

feat/new-api → main

[X]

Process-as-code.

8. Kenapa branch policy CI belum sama dengan branch protection?

CI bisa fail. Tapi platform permission menentukan apakah merge benar-benar blocked/bypass-able. GitHub Ruleset/Branch Protection idealnya enforce:

- require PR;
- require checks;
- restrict direct push.

Jadi:

```
CI policy
→ machine-check expected flow

platform ruleset
→ governance enforcement
```

9. Job 2 - Python + docs quality

Flow:

```
checkout
↓
install uv
↓
install Python
↓
uv sync --group docs
↓
Ruff
↓
pytest
↓
MkDocs strict build
```

Mari bedah.

10. Checkout

Runner GitHub fresh. Dia belum punya repo. Checkout ambil revision yang sedang diuji. Ini basic tapi lineage-critical. CI result attach ke exact commit.

11. setup-uv

Workflow pin version:

```
0.12.15
```

Kenapa pin tool version? Supaya behavior CI tidak berubah random hanya karena action/tool latest release. Dependencies/tooling juga bagian reproducibility.

12. uv python install + sync

Runner install Python sesuai project. Lalu dependency sync. Fresh runner useful karena expose hidden dependency. Kalau local code cuma works karena package global yang lupa dicatat, CI bisa fail. Itu good failure.

13. Ruff

```
bash
uv run ruff check src api scripts tests
```

Ruff catch style/static issue. Kita pernah benar-benar kena import ordering issue saat audit. Itu contoh CI memberi feedback real, bukan decorative.

14. pytest

```
bash
uv run pytest -q
```

Behavior test. Kalau feature leakage regression atau API schema issue, test seharusnya catch known failure mode. CI green memberi confidence, bukan mathematical proof zero bugs.

15. MkDocs strict build

```
bash
uv run --group docs mkdocs build --strict
```

Kenapa docs masuk quality? Karena workshop docs adalah product. Broken navigation bisa merusak participant experience sama seperti code bug merusak API. Strict build memaksa warning tertentu jadi failure.

16. Job 3 - Docker build matrix

Matrix target:

```
api
mlflow
airflow
```

Same job template dijalankan per target. Kenapa matrix? Tanpa matrix YAML akan duplicate.

```
job api
job mlflow
job airflow
```

Matrix lebih maintainable.

17. fail-fast false

Matrix punya:

```
fail-fast: false
```

Kalau API image fail, MLflow/Airflow builds tetap boleh selesai. Kenapa useful? Kita dapat full picture:

```
api [X]
mlflow [OK]
airflow [OK]
```

daripada first failure cancel semua. Debugging lebih informative.

18. Docker Compose config validation

```
bash
docker compose config --quiet
```

Ini catch invalid Compose config sebelum runtime. Python tests tidak tahu YAML networking/volume config valid atau tidak. Different artifact, different check.

19. Docker Buildx + cache

Image build pakai GitHub Actions cache. Cache scope per target. Kenapa? API layer cache dan Airflow layer cache punya content pattern beda. Cache mempercepat repeated build. Tapi remember:

Cache is optimization, not correctness dependency.

Clear cache harus tetap build.

20. Buat practice branch

Kita bisa demo dengan docs branch kecil.

```
bash
git switch develop
git pull
git switch -c docs/workshop-practice
```

Edit satu harmless docs paragraph.

```
bash
git add .
git commit -m "docs: practice contribution flow"
git push -u origin docs/workshop-practice
```

Open PR:

```
docs/workshop-practice
↓
develop
```

21. Sebelum buka Actions, predict job result

Kalau perubahan hanya Markdown:

- branch policy?
- Python quality?
- docs build?
- Docker builds?

Semua tetap jalan sesuai current workflow. Question:

“Kenapa Docker build masih jalan padahal cuma docs?”

Current workflow simple dan conservative. Future optimization bisa pakai path filters supaya docs-only PR skip expensive Docker builds. Tradeoff:

```
simple + safe
vs
optimized runner cost
```

22. Observe GitHub Actions UI

Buka workflow run. Cari jobs:

```
docs/workshop-practice
```

Klik quality job. Read steps sequentially. Jangan hanya lihat final green.

23. Controlled failure idea

Presenter bisa punya demo branch dengan intentional docs nav typo atau Ruff issue. Goal:

lihat quality gate fail sebelum merge.

Kemudian fix, push lagi, lihat CI green. Ini membuat value automation tangible. Jangan lakukan di main.

24. Develop → main

Setelah feature/docs PR squash merged ke develop dan develop green:

```
develop
↓ PR
main
```

Branch policy menerima hanya develop ke main. Main = stable release.

25. Open cd.yml

CD trigger:

```
workflow_run
→ CI completed
```

Tapi publish job punya condition. Dia hanya publish kalau:

```
CI conclusion = success
event = push
head branch = main
```

Kenapa tidak PR? Karena PR candidate belum stable release.

26. Checkout tested commit

CD checkout:

```
workflow_run.head_sha
```

Ini sangat important. CI test SHA A. CD build SHA A. Bukan random current latest. Tested = published.

27. Login GHCR

Workflow pakai:

```
GITHUB_TOKEN
```

dan permission:

```
packages: write
```

No personal password committed.

28. Metadata tags

Docker metadata action produce:

```
sha-<commit>
latest
```

SHA untuk traceability. latest convenience pointer. Coba answer:

“Kalau mau rollback deterministic, pilih yang mana?”

SHA.

29. Build and push

Per target:

```
api
mlflow
airflow
```

Build exact Dockerfile target lalu push tags ke GHCR. Sekarang container registry punya deployable artifact.

30. Kenapa belum SSH/Kubernetes deploy?

Karena workshop belum punya real production target. Tambahkan fake deploy command akan membuat peserta mengira deployment selesai hanya dengan satu baris. Real deployment needs:

- credentials;
- network;
- TLS;
- rollout;
- health;
- rollback;
- scaling;
- secrets.

Kita berhenti jujur di Continuous Delivery.

31. Application release vs model release

Setelah CI/CD, ingat ada dua lifecycle.

```
GitHub Actions  
→ application artifact
```

```
MLflow Registry  
→ model artifact
```

Main image bisa tetap sama sementara champion berubah. Atau code image berubah sementara champion tetap. Ini decoupling yang penting.

32. Mini scenarios

Scenario A

pytest fail. Apakah image publish? No, CI tidak success.

Scenario B

PR green tapi belum merge main. Publish latest? No.

Scenario C

main push CI success. Publish? Yes.

Scenario D

champion alias berubah tanpa code commit. Apakah GitHub Actions harus build image? Tidak necessarily. Model lifecycle independent.

Checkpoint

1. Workflow/job/step beda apa?
2. Trigger CI apa saja?
3. Branch policy rules?
4. CI policy dan branch protection beda apa?
5. Fresh runner kenapa useful?
6. Ruff, pytest, MkDocs, Docker masing-masing test layer apa?
7. Matrix build solve apa?
8. fail-fast false memberi benefit apa?
9. Cache role-nya apa?

10. develop → main kenapa separate?
11. CD condition publish kapan?
12. Kenapa checkout tested SHA?
13. latest vs SHA tag?
14. Kenapa workshop berhenti di Delivery?
15. Application release dan model release beda apa?

Kalau kalian bisa baca workflow dari event sampai artifact dan explain why, GitHub Actions nggak lagi kelihatan seperti YAML ritual.

Step 9 - Prometheus, Grafana, dan evaluasi

Sekarang stack kita sudah hidup. FastAPI bisa menerima prediction request. Prometheus bisa scrape metric. Grafana bisa render dashboard. Tapi ada satu hal yang sering banget bikin bingung pas pertama belajar monitoring:

“Kok dashboard-nya kosong? Berarti Grafana error ya?”

Belum tentu. Dashboard hanya bisa menampilkan signal yang memang sudah diproduksi oleh system. Kalau belum ada traffic, belum ada prediction, atau ground truth belum datang, beberapa panel memang seharusnya kosong atau zero. Di step ini kita nggak cuma mau bikin graph bergerak. Kita mau ngerti **chain yang membuat graph itu bisa bergerak**.

Goal

Setelah selesai, kalian harus bisa menjelaskan dua chain berbeda. Operational monitoring:

```
HTTP request
↓
FastAPI middleware
↓
Prometheus metric
↓
Prometheus scrape
↓
Grafana panel
```

Model performance monitoring:

```
prediction
↓
prediction log
↓
ground truth datang belakangan
↓
evaluation
↓
performance summary
↓
FastAPI metrics endpoint
↓
Prometheus
↓
Grafana
```

Kenapa dua chain? Karena latency bisa diketahui langsung saat request selesai, tapi prediction accuracy baru diketahui setelah actual target tersedia.

1. Buka Grafana sebelum generate traffic

Open:

```
http://localhost:3000
```

Cari dashboard:

```
ML0ps / Taxi Demand ML0ps
```

Lihat panel. Beberapa mungkin:

```
0
no data
flat
```

Jangan buru-buru restart Grafana. Tanya:

“Event yang mau diukur memang sudah terjadi belum?”

Kalau belum ada prediction request, prediction counter zero itu benar. Kalau belum ada evaluation, recent MAE belum punya value juga benar. Monitoring yang bagus tidak mengarang data supaya dashboard terlihat cantik.

2. Sebelum Grafana, cek Prometheus target

Open:

```
http://localhost:9090
```

Masuk ke Targets. Target API harus:

```
UP
```

Kenapa kita cek ini di awal? Karena dependency-nya:

```
FastAPI /metrics
↓
Prometheus scrape
↓
Grafana
```

Kalau Prometheus bahkan nggak bisa scrape FastAPI, debug Grafana dulu itu salah layer. Ini recurring debugging principle:

Start from the earliest broken dependency, not from the prettiest UI.

3. Buka raw /metrics

Open:

```
http://localhost:8000/metrics
```

Output-nya raw text. Kelihatan nggak friendly. Normal. Endpoint ini primarily buat Prometheus. Cari:

```
taxi_api_requests_total
taxi_api_request_duration_seconds
taxi_predictions_total
```

Coba observe label-nya. Misalnya request metric punya dimension method, path, status. Sekarang kalian bisa connect:

```
Python metric definition
↓
/metrics exposition
↓
Prometheus series
```

4. Predict dulu apa yang akan terjadi sebelum generate traffic

Kita akan run:

```
bash
uv run python scripts/generate_api_traffic.py \
  --date 2025-01-28 --start-hour 17 --end-hour 18
```

Sebelum Enter, coba jawab:

1. Script ini call predictor langsung atau lewat HTTP?
2. Berapa jam target yang dikirim?
3. Apakah prediction log akan bertambah?
4. Apakah API request counter akan bertambah?
5. Apakah latency histogram akan punya observation baru?

Expected:

```
HTTP
2 target hours
yes
yes
yes
```

Baru run.

5. Apa yang dilakukan generate_api_traffic.py?

Actual script:

```
load Manhattan zone IDs
↓
for hour 17..18
↓
for each zone
↓
POST /predict
↓
count success / failed
```

Jadi jumlah request kira-kira:

```
number_of_manhattan_zones × 2
```

Ini bukan random fake metric injection. Kita benar-benar exercise FastAPI serving path.

6. Watch success dan failed

Terminal akan report:

```
success
failed
target api
```

Kalau failed banyak, jangan lanjut seolah semuanya baik. Read response. Possible reason:

- champion belum ada;
- history tidak lengkap;

- API URL salah;
- service down;
- request invalid.

Operational monitoring demo harus mulai dari request yang memang berhasil.

7. Kenapa script ini berbeda dari `replay_predictions.py`?

Ini penting.

`generate_api_traffic.py`

```
script
↓ HTTP
FastAPI
↓
predictor
```

Effect:

```
request counter [OK]
latency histogram [OK]
prediction counter [OK]
prediction log [OK]
```

`replay_predictions.py`

```
script
↓
predictor directly
```

Effect:

```
request counter [X]
HTTP latency [X]
prediction log [OK]
```

Kenapa direct replay masih useful? Kalau kita mau batch historical evaluation, HTTP overhead bisa unnecessary. Jadi:

```
API traffic generator
→ operational monitoring demo

direct replay
→ batch model evaluation
```

Satu project bisa punya dua path yang sengaja berbeda.

8. Query raw counter

Di Prometheus query:

```
taxi_api_requests_total
```

Observe value. Ini cumulative counter. Question:

“Kalau total requests sudah 1000, apakah traffic saat ini tinggi?”

Belum tentu. 1000 bisa terkumpul selama seminggu. Makanya counter sendiri belum jawab current traffic speed.

9. Query rate

Try:

```
sum(rate(taxi_api_requests_total[1m]))
```

Sekarang kita estimate seberapa cepat counter bertambah dalam recent one-minute window. Query di atas mencakup seluruh path API, termasuk request ke `/metrics` dari Prometheus. Untuk membaca traffic prediksi seperti panel **Prediction Request Rate** di Grafana, filter path `/predict`:

```
sum(rate(taxi_api_requests_total{path="/predict"}[1m]))
```

Request scrape atau health check tidak membuktikan peserta sudah mengirim prediksi. Untuk checkpoint traffic, gunakan query yang memfilter `/predict` dan cek prediction log. Analogy:

```
counter
→ odometer

rate(counter)
→ speedometer
```

Odometer bilang total perjalanan. Speedometer bilang current speed. Ini salah satu mental model Prometheus paling useful.

10. Latency dan p95

Dashboard punya p95 latency. Kenapa p95? Bayangin:

```
95 requests = 50 ms
5 requests = 4 sec
```

Kalau cuma average, tail problem bisa kelihatan lebih kecil. p95 roughly bertanya:

"95% request selesai di bawah sekitar berapa lama?"

Tail latency penting karena user yang unlucky tetap merasakan slow response.

11. Refresh Grafana

Wait beberapa detik buat:

```
FastAPI metric update
↓
Prometheus scrape
↓
Grafana refresh
```

Sekarang request rate dan latency harus mulai punya signal. Kalau belum, debugging order:

```
generate script success?
↓
/metrics value changed?
↓
Prometheus target UP?
↓
Prometheus query has data?
↓
Grafana datasource/panel query?
```

Jangan random restart seluruh stack.

12. Sekarang pindah ke model monitoring

Operational metric bergerak. Tapi kita belum menjawab:

“Prediction-nya bagus nggak?”

Untuk itu, buka:

```
data/monitoring/predictions.jsonl
```

Lihat beberapa entries. Fields penting:

```
logged_at
zone_id
target_datetime
predicted_trip_count
model_version
run_id
```

13. Kenapa model_version harus ikut di prediction log?

Bayangin setengah hari pertama champion v1. Siang hari champion pindah v2. Kalau log cuma simpan:

```
target
prediction
```

nanti kita nggak tahu prediction buruk berasal dari model mana. Model version adalah bagian dari prediction lineage.

14. Ground truth harus tersedia

Prediction untuk Jan 28 hanya bisa dievaluate kalau processed actual demand Jan 28 tersedia. Check:

```
data/processed/demand/2025-01-28.parquet
```

Kalau file belum ada, run daily replay Jan 28 dulu. Ini penting secara semantics:

```
missing ground truth
≠
zero error
```

Kalau actual belum ada, kita belum tahu model benar atau salah.

15. Run `evaluate_predictions.py`

```
bash
uv run python scripts/evaluate_predictions.py
```

Actual flow:

```
resolve current champion
↓
ambil champion validation MAE
↓
load prediction log
↓
filter prediction by champion model version
↓
join with ground truth
↓
calculate absolute/squared errors
↓
take recent evaluation window
↓
calculate MAE/RMSE
↓
compare threshold
↓
write artifacts
```

Banyak ya? Makanya model monitoring bukan sekadar satu Prometheus query.

16. Kenapa filter by model version?

Suppose:

```
v1 predictions = 300
v2 predictions = 200
```

Kalau dicampur:

```
recent MAE
```

jadi mixture dua model. Kalau kita sedang evaluate champion v2, summary tersebut misleading. Lineage membantu monitoring tetap model-specific.

17. Baca terminal output

Output punya bentuk seperti:

```
model version: ...
evaluated: ...
reference MAE: ...
recent MAE: ...
threshold MAE: ...
retrain recommended: ...
```

Exact numeric values tergantung actual run. Jangan hafal angka contoh dari docs. Yang perlu dipahami relationship:

```
recent_mae
vs
threshold_mae
```

dan:

```
evaluation_count
vs
min_samples
```

18. Buka evaluations.parquet

Path:

```
data/monitoring/evaluations.parquet
```

Optional inspect:

```
bash
uv run python - <<'PY'
import pandas as pd
df = pd.read_parquet('data/monitoring/evaluations.parquet')
print(df.head())
print(df.shape)
PY
```

Cari:

```
predicted_trip_count
actual_trip_count
absolute_error
squared_error
```

Ini row-level evidence di balik summary. Kalau MAE terlihat aneh, kita bisa inspect individual errors.

19. Buka performance_summary.json

```
data/monitoring/performance_summary.json
```

Fields:

```
evaluation_count
recent_mae
recent_rmse
reference_mae
threshold_mae
retrain_recommended
latest_target_datetime
model_version
```

Question:

“Kalau retrain_recommended false, monitoring gagal?”

No. Bisa berarti champion masih healthy. No action adalah valid outcome.

20. Bagaimana summary masuk ke Prometheus?

FastAPI metrics endpoint refresh model-performance gauge dari summary file saat scrape. Flow:

```
performance_summary.json
↓
FastAPI /metrics
↓
Prometheus scrape
↓
taxi_model_recent_mae
taxi_model_reference_mae
taxi_model_retrain_recommended
```

Grafana tidak membaca JSON langsung. Ini dependency chain penting.

21. Refresh Grafana lagi

Setelah evaluation dan next scrape, panel recent MAE/reference MAE seharusnya mulai muncul kalau ada evaluated samples. Kalau kosong:

```
summary recent_mae null?
↓
/metrics expose gauge?
↓
Prometheus query punya value?
↓
Grafana panel?
```

Debug step by step.

22. Scenario game - diagnose layer-nya

Scenario A

```
p95 latency = 60 ms
HTTP success normal
recent MAE jauh naik
```

Interpretasi:

```
service health
[OK]

model quality
[X]
```

Potential investigation:

- data quality;
- distribution shift;
- model aging;
- retraining.

Restart Grafana bukan solution.

Scenario B

```
recent MAE normal  
p95 latency = 8 sec  
timeout naik
```

Interpretasi:

```
model quality  
[OK]  
  
service performance  
[X]
```

Retraining model bukan first action.

Scenario C

```
request rate zero  
all service healthy
```

Mungkin memang nggak ada traffic. Zero traffic bukan otomatis incident.

Scenario D

```
Prometheus target DOWN  
Grafana stale
```

Investigate API/network/scrape config before dashboard.

23. Cardinality thought experiment

Bayangin kita tambahkan label:

```
request_id = unique UUID
```

Setiap request menghasilkan combination baru. Prometheus bisa punya sangat banyak series. Itu high cardinality. Bandingkan:

Bounded-ish labels

```
method  
path category  
HTTP status  
model version
```

Dangerous unbounded labels

```
request_id  
user_id  
free text  
full timestamp  
dynamic URL ID
```

Detailed individual info cocok di logs/events, bukan metric labels.

24. Unknown vs zero

Ini subtle tapi penting. Case ground truth belum ada. Wrong:

```
error = 0
```

Itu berarti model perfect. Correct:

```
error = unknown / not evaluated
```

Dalam data system:

Missing dan zero punya semantics berbeda.

Kalau kita salah represent missing sebagai zero, monitoring terlihat lebih bagus dari reality.

25. Monitoring detects symptom, not necessarily root cause

MAE naik adalah symptom. Possible causes:

- data drift;
- data corruption;
- feature pipeline bug;
- extraordinary event;
- model aging;
- target definition change.

Monitoring bilang:

"Something deserves investigation."

Bukan:

"Pasti retrain."

Ini kenapa Step 10 masih punya governance boundary.

26. Experiment dengan policy

Script support:

```
--degradation-multiplier
--min-samples
--recent-limit
```

Untuk learning, coba:

```
bash
uv run python scripts/evaluate_predictions.py --min-samples 10
```

Lihat recommendation. Lalu compare default 100. Pertanyaan:

"Mana yang lebih benar?"

Tidak ada universal answer. Policy tergantung risk, noise, business cost, dan data volume. Workshop goal-nya memahami sensitivity.

27. Propagation delay

Suppose performance_summary baru diupdate. Grafana mungkin belum instant berubah. Chain:

```
file updated
↓
next /metrics scrape
↓
Prometheus stores sample
↓
next Grafana refresh
↓
panel changes
```

Ada small propagation delay. Jadi jangan spam refresh dan conclude failure dalam 100 ms. Observability system juga punya latency.

28. Monitoring chain recap

Operational:

```
HTTP request
↓
middleware
↓
metrics
↓
Prometheus
↓
Grafana
```

Model:

```
prediction
↓
prediction log
↓
actual later
↓
evaluation
↓
summary
↓
metrics
↓
Prometheus
↓
Grafana
```

Model chain lebih panjang karena ground truth delayed.

Direct replay: apa bedanya

Coba jawab tanpa lihat docs:

1. Kenapa empty dashboard belum tentu error?
2. Prometheus target UP artinya apa?
3. generate_api_traffic beda apa dengan replay_predictions?
4. Counter beda apa dengan rate?
5. Kenapa p95 useful?
6. Kenapa prediction log simpan model_version?

7. Kenapa ground truth delayed?
8. Missing ground truth kenapa tidak boleh dianggap zero error?
9. Kenapa evaluation filter model version?
10. performance_summary sampai Grafana lewat chain apa?
11. retrain_recommended false bisa jadi success kenapa?
12. MAE naik berarti pasti drift nggak?
13. High-cardinality label itu apa?
14. Kalau panel kosong, debugging order yang masuk akal bagaimana?
15. Kenapa dashboard punya propagation delay?

Kalau semua kebayang, kalian bukan cuma “bisa pakai Grafana”. Kalian ngerti observability architecture di balik panel. Next kita tutup loop dengan retraining.

Urutan demo untuk dashboard repository ini

Kalau data, model, dan service belum disiapkan, mulai dari Quickstart Grafana dari kondisi kosong. Quickstart menyiapkan training, champion, HTTP traffic, dan evaluasi. Setelah prediction API menghasilkan log dan actual demand tersedia, jalankan:

```
bash
uv run python scripts/evaluate_predictions.py
```

Periksa evaluation_status di output dan data/monitoring/performance_summary.json. Untuk memperagakan status sample belum cukup, jalankan ulang dengan `--min-samples 100000` setelah ada prediction yang cocok dengan actual, lalu kembalikan ke default dengan menjalankan command tanpa opsi tersebut. Tanpa matched sample, status tetap belum dievaluasi. Dashboard memakai dua identitas model: serving version dari prediction API terakhir dan evaluated version dari summary MAE. Keduanya boleh berbeda sesudah promotion. Dashboard juga memakai dua clock: waktu nyata job evaluasi dan tanggal target replay historis di timezone America/New_York.

Step 10 - Retraining dan promotion

Ini final hands-on lifecycle kita. Sebelumnya kita sudah melakukan:

```
data
↓
feature engineering
↓
snapshot
↓
training
↓
tracking
↓
registry
↓
serving
↓
monitoring
```

Sekarang pertanyaan terakhir:

“Kalau monitoring bilang champion mulai memburuk, apa yang seharusnya system lakukan?”

Jawabannya bukan langsung:

“Replace production model sekarang!”

Kita ingin lifecycle yang lebih controlled:

```
detect
↓
decide
↓
retrain if needed
↓
create challenger
↓
review
↓
promote
```

Di sini kita benar-benar menutup feedback loop MLOps.

Goal

Setelah step ini kalian harus bisa menjelaskan:

- reference MAE datang dari mana;
- kenapa threshold relative;
- kenapa minimum sample diperlukan;
- kenapa monitoring run bisa success tanpa retrain;
- bagaimana cutoff retraining ditentukan;
- kenapa snapshot baru dibuat;
- kenapa runtime worker tidak auto-commit Git/DVC;
- kenapa validation window ikut maju;

- kenapa baseline tetap jadi guardrail;
- challenger vs champion;
- automatic retraining vs automatic promotion;
- bagaimana serving akhirnya detect champion baru.

Kalau semua ini clear, kalian sudah paham lifecycle decision, bukan cuma command-nya.

1. Jangan trigger DAG dulu - baca evidence-nya

Setelah step ini:

- performance summary dipahami,
- monitoring DAG dijalankan,
- healthy path dipahami,
- degraded path dipahami,
- challenger lifecycle dipahami,
- promotion tetap explicit.

1. Buka summary

```
data/monitoring/performance_summary.json
```

Cari:

```
evaluation_count
recent_mae
recent_rmse
reference_mae
threshold_mae
retrain_recommended
latest_target_datetime
model_version
```

Kenapa baca manual dulu? Karena Airflow tidak seharusnya menjadi black box decision maker. Kita harus tahu input decision sebelum automation act.

2. Reference MAE datang dari mana?

Monitoring code memanggil champion reference dari MLflow. Concept:

```
champion alias
↓
model version
↓
source run
↓
validation MAE
```

Jadi reference bukan angka random yang ditulis lagi di monitoring script. Lineage dipakai. Kalau champion berubah, reference performance juga bisa berubah sesuai source run model baru.

3. Hitung threshold sendiri

Suppose:

```
reference_mae = 10  
degradation_multiplier = 1.25
```

Threshold:

```
10 × 1.25 = 12.5
```

Sekarang:

```
recent_mae = 13.2  
evaluation_count = 250  
min_samples = 100
```

Decision:

```
13.2 > 12.5  
AND  
250 >= 100  
↓  
retrain_recommended = true
```

Simple rule. Tapi dia punya reasoning yang jelas.

4. Kenapa threshold relative?

Bayangin dua ML task. Task A:

```
normal MAE sekitar 0.8
```

Task B:

```
normal MAE sekitar 120
```

Kalau kita pakai universal:

```
retrain if MAE > 20
```

Task A telat banget bereaksi. Task B mungkin selalu retrain. Relative threshold anchor ke known champion quality. Masih simplification, tapi lebih portable secara semantics.

5. Kenapa multiplier 1.25?

Jangan anggap 1.25 angka sakral. Dia workshop default supaya mudah dijelaskan. Production policy seharusnya mempertimbangkan:

- business tolerance;
- noise;
- sample volume;
- cost of wrong prediction;
- cost of retraining;
- seasonality;
- segment behavior.

Nilai threshold adalah policy decision, bukan universal law Machine Learning.

6. Minimum sample

Case:

```
recent_mae = 30
evaluation_count = 3
```

Haruskah retrain? Belum tentu. Tiga sample sangat mudah dipengaruhi outlier. Makanya kita punya:

```
min_samples = 100
```

Decision butuh:

```
magnitude of degradation
+
enough evidence
```

Ini seperti kita nggak mau ganti resep restoran cuma karena satu customer bilang kurang asin. Kita cari pattern.

7. recent_limit

Misalnya:

```
recent_limit = 500
```

Kenapa tidak pakai semua historical prediction? Karena monitoring ingin menjawab:

“Bagaimana behavior terbaru?”

Kalau six months old performance masuk dengan weight besar, recent degradation bisa tertutup average lama. Recent window memberi sensitivity ke current behavior.

8. Open Airflow monitoring DAG

Buka:

```
taxi_model_monitoring
```

Graph:

```
evaluate_model
↓
maybe_retrain
```

Kelihatannya cuma dua task. Tapi second task punya conditional lifecycle cukup panjang. Sebelum trigger, lihat parameter form. Expected default:

```
degradation_multiplier = 1.25
min_samples = 100
recent_limit = 500
```

9. Predict outcome sebelum Trigger

Lihat performance_summary. Kalau:

```
retrain_recommended = false
```

coba predict:

“Apa maybe_retrain akan train model?”

Expected:

```
no
```

Kalau true, expect:

```
snapshot
↓
training
↓
MLflow run
↓
candidate gate
↓
challenger
```

Baru trigger. Again: hypothesis before clicking.

10. Task `evaluate_model`

Actual DAG call:

```
evaluate_champion_performance
```

Function ini:

```
resolve champion reference
↓
load prediction log
↓
build evaluation table
↓
summarize performance
↓
save evaluation parquet
↓
save summary JSON
↓
return summary
```

Jadi Airflow task tidak duplicate calculation logic. Reusable logic tetap di src/monitoring dan orchestration module.

11. Healthy path

Kalau `monitoring_result`:

```
retrain_recommended = false
```

function return:

```
status = not_needed
reason = performance_within_threshold
```

DAG success. Pause di sini. Ini penting banget:

Automation success tidak selalu berarti action executed.

Kadang decision paling tepat adalah mempertahankan champion. Kalau system selalu retrain setiap monitoring run, condition check jadi pointless.

12. No evaluated predictions path

Ada case:

```
latest_target_datetime = null
```

Misalnya belum ada prediction yang ground truth-nya available. Correct behavior:

```
not_needed
reason = no_evaluated_predictions
```

Bukan force training. Unknown evidence bukan signal degradation.

13. Degraded path - overview

Kalau recommendation true:

```
latest_target_datetime
↓
derive cutoff_date
↓
create retraining snapshot
↓
train candidate
↓
log MLflow stage retraining-validation
↓
compare with naive baseline
↓
register challenger if eligible
```

Mari bedah detailnya.

14. latest_target_datetime menjadi cutoff

Suppose latest evaluated target:

```
2025-02-10 23:00
```

Code ambil date:

```
2025-02-10
```

Kenapa cutoff berdasarkan evaluated target? Karena kita tahu ground truth legitimate tersedia sampai logical point itu. Historical replay harus menjaga temporal integrity. Kita nggak mau retraining memakai future data yang belum seharusnya available.

15. Create retraining snapshot

Feature dataset aktif bisa terus berkembang. Retraining freeze data sampai cutoff. Output misalnya:

```
data/snapshots/training/taxi_demand_2025-02-10.parquet
```

Metadata snapshot:

```
name
rows
zones
sha256
```

Kenapa hash lagi? Supaya MLflow run punya identity data yang kuat.

16. Initial DVC snapshot vs runtime retraining snapshot

Ini nuance yang bagus buat dibahas. Initial training:

```
DVC repro
↓
reproducible training snapshot stage
```

Runtime retraining:

```
Airflow
↓
create snapshot for latest cutoff
↓
log snapshot fingerprint to MLflow
```

Kenapa Airflow tidak otomatis:

```
git add
git commit
dvc commit
git push
```

Karena runtime worker jangan diam-diam mutate source repository sebagai side effect training. Source-control workflow dan runtime artifact workflow punya responsibility beda. Production matang biasanya save runtime snapshot ke versioned object storage/artifact system. Workshop kita log frozen file + SHA ke MLflow.

17. Train candidate dengan same experiment logic

Retraining reuse:

```
run_mlflow_experiment
```

Stage tag:

```
retraining-validation
```

Kenapa reuse? Supaya initial training dan retraining tidak pelan-pelan diverge. Kalau initial flow calculate metric satu cara dan retraining flow punya implementation beda, comparison menjadi susah dipercaya. One reusable experiment function menjaga consistency.

18. Moving validation window

Split model sekarang dinamis. Rule:

last 5 days of snapshot = validation.

Initial snapshot sampai Jan 26:

```

training
→ earlier model-ready rows through Jan 21

validation
→ Jan 22-26

```

Snapshot retraining sampai Feb 10:

```

training
→ through Feb 5

validation
→ Feb 6-10

```

Kenapa validation ikut maju? Karena kita ingin evaluate candidate pada recent unseen period. Kalau February model masih diuji hanya January holdout, kita belum menjawab apakah dia generalize ke recent behavior.

19. Kenapa validation tidak pakai random split?

Sama seperti initial training. Time-series problem punya temporal order. Random split bisa membuat future-like rows masuk training sementara older rows masuk validation. Kita ingin simulation:

```

learn from past
↓
validate on later period

```

Retraining tidak mengubah prinsip temporal validation.

20. Baseline gate

Setelah candidate train, experiment result punya:

```
model_beats_baseline
```

register_challenger check ini. Kalau false:

```

status = not_registered
reason = model_did_not_beat_baseline

```

Kenapa baseline masih dibutuhkan? Karena data baru bisa membuat complex model perform buruk. Tidak ada guarantee gradient boosting selalu menang selamanya. Baseline adalah guardrail setiap cycle.

21. Apakah beat baseline cukup buat jadi champion?

No. Ini distinction penting.

```

beat naive baseline
→ candidate layak dipertimbangkan

beat current champion under all relevant criteria
→ belum tentu

```

Misalnya candidate:

```
MAE sedikit lebih baik
latency jauh lebih buruk
```

atau:

```
overall MAE bagus
zone penting tertentu jelek
```

Workshop belum punya semua governance check itu. Makanya automatic step berhenti di challenger.

22. Challenger alias

Kalau candidate eligible:

```
register model version
↓
set alias challenger
```

Sekarang Registry punya candidate baru yang traceable ke:

```
run ID
snapshot
metrics
params
```

Review bisa dilakukan dengan evidence.

23. Review sebelum promotion

Minimal inspect:

```
candidate validation MAE
candidate RMSE
baseline MAE
dataset cutoff
dataset SHA
source run
current champion reference
recent champion production-like MAE
```

Kalau cuma lihat version baru lebih besar nomornya, itu bukan model review. Version number tidak imply quality.

24. Promote explicit

Kalau approved:

```
bash
uv run python scripts/promote_model.py --version "VERSION"
```

Registry:

```
champion
↓
new version
```

Kita tidak edit API source code.

25. Bagaimana API detect champion baru?

RegistryModelLoader periodically refresh metadata. Suppose current cache:

```
v3
```

Promotion:

```
champion → v5
```

After refresh interval:

```
query champion
↓
version changed?
↓ yes
load v5
↓
cache v5
```

Future requests use v5. Ini close loop Registry → Serving.

26. Policy sensitivity experiment

Di local workshop, kita boleh experiment parameter. Example:

```
multiplier = 1.25
vs
multiplier = 1.05
```

1.05 lebih sensitive. Apakah otomatis lebih bagus? No. Terlalu sensitive:

```
small noise
↓
retrain often
↓
compute waste
↓
many challengers
↓
review fatigue
```

Terlalu loose:

```
model degraded
↓
system wait too long
↓
bad predictions persist
```

Threshold adalah control policy tradeoff.

27. Thermostat analogy

Bayangin thermostat rumah. Kalau threshold terlalu sensitive:

```
temperature berubah 0.1°  
↓  
AC on  
↓  
off  
↓  
on  
↓  
off
```

System oscillate. Monitoring policy juga bisa begitu:

```
tiny MAE fluctuation  
↓  
evaluate  
↓  
retrain  
↓  
tiny fluctuation  
↓  
retrain lagi
```

Kita ingin stable decision, bukan hyperactive automation.

28. Retraining tidak solve bad data

Suppose MAE naik karena processed data corruption. Kalau response kita:

```
bad data  
↓  
retrain  
↓  
model learns bad data
```

kita memperburuk problem. Mature lifecycle biasanya punya data-quality gate juga. Jadi retrain recommendation berarti condition for candidate refresh reached, bukan root cause pasti model lama.

29. Batch retraining vs continual learning

Project kita:

```
batch retraining
```

Flow:

```
collect newer data  
↓  
freeze snapshot  
↓  
train new model  
↓  
replace only after review
```

Continual/online learning:

```
new observations
↓
incrementally update model parameters
```

Dua paradigm beda. Pipeline kita berulang bukan berarti continual learning.

30. Full lifecycle tanpa nama tools

Sekarang coba cerita full project tanpa sebut tool. Contoh:

“Kita menerima data historical secara bertahap, aggregate jadi demand per zone-hour, membangun feature hanya dari masa lalu, membekukan training input, mencatat experiment, memilih model approved, expose prediction lewat service, menyimpan prediction, menunggu actual, menghitung recent error, dan kalau performance cukup memburuk dengan evidence yang cukup, kita train candidate baru lalu review sebelum production replacement.”

Kalau narrative ini sudah bisa dijelaskan, baru mapping tool:

```
data orchestration
→ Airflow

snapshot
→ DVC

experiment + registry
→ MLflow

serving
→ FastAPI

runtime
→ Docker

code lifecycle
→ GitHub Actions

metrics
→ Prometheus

dashboard
→ Grafana
```

Ini lebih powerful daripada hafal list tools dulu.

Final challenge

Scenario 1

Recent MAE naik, sample cuma 8. Action?

```
wait / insufficient evidence
```

Scenario 2

Recent MAE naik, sample 500, threshold exceeded. Action?

```
create retraining candidate
```

Scenario 3

Candidate beat baseline. Auto champion?

```
no  
review first
```

Scenario 4

Candidate promoted to champion. Edit FastAPI version constant?

```
no  
serving resolves alias
```

Scenario 5

MAE naik karena data corrupt. Retrain directly?

```
investigate/fix data first
```

Kalau reasoning ini clear, feedback loop-nya benar-benar dipahami.

Final checkpoint

1. Reference MAE datang dari mana?
2. Kenapa multiplier bukan universal truth?
3. Minimum samples solve apa?
4. recent_limit buat apa?
5. Healthy monitoring run boleh no-op kenapa?
6. latest_target_datetime dipakai buat apa?
7. Kenapa retraining snapshot baru diperlukan?
8. Kenapa runtime Airflow tidak auto Git commit?
9. Kenapa validation window bergerak?
10. Kenapa time split tetap chronological?
11. Baseline gate solve apa?
12. Beat baseline kenapa belum otomatis champion?
13. Challenger dan champion beda apa?
14. Threshold terlalu sensitive efeknya apa?
15. Threshold terlalu loose efeknya apa?
16. Kenapa retraining tidak solve corrupt data?
17. Batch retraining vs continual learning?
18. Bagaimana API detect champion baru?

Kalau kalian bisa jawab dengan bahasa sendiri, congratulations peers - kalian sudah bukan cuma mengikuti command MLOps. Kalian sudah mulai memahami **kenapa lifecycle-nya dirancang seperti ini**.

Lampiran A - Quickstart dashboard dari nol

Panduan ini menjalankan alur **data** → **training** → **champion** → **HTTP traffic** → **evaluasi** → **Grafana** secara manual. Cocok untuk demo pertama atau mengecek dashboard sebelum sesi workshop. Airflow belum diperlukan. Untuk materi konsep, latihan, dan panduan fasilitator, baca Modul Workshop.

Sebelum mulai

- Jalankan semua command dari root repository.
- Gunakan terminal Bash di Linux, macOS, atau WSL2.
- Siapkan Git, uv, Docker Engine/Desktop yang aktif, dan Docker Compose.
- Pastikan internet tersedia untuk dependency, image, dan dataset TLC Januari.
- Panduan memakai port dan nama model default di `docker-compose.yml`.

Jika menggunakan PowerShell, sesuaikan sintaks environment variable. Command Python tetap sama. Jangan menjalankan server MLflow kedua pada port 5000.

1. Siapkan environment Python

Kalau repo belum tersedia:

```
bash
git clone https://github.com/ahmadnafi30/bcc-workshop-mlops.git
cd bcc-workshop-mlops
```

Install dependency dan cek environment:

```
bash
uv sync
uv run python --version
uv run python scripts/doctor.py
```

Hasil yang diperiksa: Python 3.11 dan semua import wajib berstatus OK. Doctor mengecek keberadaan Docker CLI; kesiapan daemon diperiksa di langkah 2.

2. Jalankan service

Siapkan folder state pada host sebelum membuat container:

```
bash
mkdir -p .mlflow/artifacts data
docker info
docker compose up -d --build grafana
docker compose ps
```

Compose ikut menjalankan dependency Grafana: Prometheus, API, dan MLflow. Kalau stack sudah berjalan, command yang sama dapat dipakai untuk menerapkan perubahan image dan konfigurasi repo.

Service	Alamat	Yang diperiksa
MLflow	<code>http://localhost:5000</code>	UI tracking dapat dibuka

Service	Alamat	Yang diperiksa
API health	http://localhost:8000/health	status bernilai ok
Swagger	http://localhost:8000/docs	Endpoint API tersedia
Prometheus	http://localhost:9090	Target taxi-api berstatus UP
Grafana	http://localhost:3000/d/taxi-demand-mlops/taxi-demand-mlops	Dashboard tersedia

API health bisa ok sementara model belum tersedia. Di tahap ini, /model-info bisa mengembalikan 503 dan dashboard masih kosong. Arahkan script host ke server MLflow Docker:

```
bash
export MLFLOW_TRACKING_URI=http://127.0.0.1:5000
```

Pertahankan terminal ini untuk langkah berikutnya. Jika membuka terminal baru, set kembali variable tersebut. Container API memakai alamat internal http://mlflow:5000 dari Compose.

3. Download dan siapkan data

```
bash
uv run python scripts/bootstrap_data.py --months 2025-01

uv run python scripts/prepare_historical_demand.py \
  --start-date 2025-01-01 --end-date 2025-01-28
```

Bootstrap mengunduh lookup zone dan data Yellow Taxi, lalu menyiapkan replay source untuk Manhattan. Pemrosesan pertama memerlukan waktu dan memori untuk membaca file bulanan.

Hasil yang diperiksa: metadata zone, replay source Januari, dan file data/processed/demand/2025-01-28.parquet tersedia. Script history juga menyiapkan hari-hari sebelumnya yang dibutuhkan oleh lag sampai 168 jam.

4. Buat feature dan snapshot training

```
bash
uv run python scripts/build_features.py \
  --start-date 2025-01-01 --end-date 2025-01-26 --force

uv run python scripts/create_training_snapshot.py \
  --cutoff-date 2025-01-26
```

Hasil yang diperiksa:

- data/features/taxi_demand_features.parquet terbentuk.
- data/snapshots/training/taxi_demand_2025-01-26.parquet terbentuk.
- Output snapshot menampilkan jumlah row, zone, rentang waktu, dan SHA256.

Cutoff training tetap Jan 26. Data Jan 27-28 disiapkan untuk inference dan ground truth, sehingga tidak ikut masuk snapshot initial training. Panduan cepat ini membuat snapshot langsung. Pada sesi reproducibility di modul workshop, snapshot dikelola melalui stage DVC.

5. Train dan catat model run ID

```
bash
uv run python scripts/train_with_mlflow.py
```

Training menghasilkan dua run: baseline naive 24 jam dan model HGB. Contoh struktur output berikut menggunakan placeholder, bukan nilai hasil training:

```
baseline -> MAE ..., run BASELINE_RUN_ID
model    -> MAE ..., run MODEL_RUN_ID
model uri: runs:/MODEL_RUN_ID/model
```

Salin run ID dari baris model, bukan baseline. Buka MLflow untuk membandingkan MAE/RMSE, dataset SHA256, parameter, dan artifact model. Model HGB menggunakan serialization skops dengan daftar trusted types terbatas yang didefinisikan dalam repo. Jika environment lama belum mendukung parameter ini, jalankan kembali `uv sync` sesuai `pyproject.toml`.

6. Register dan promote ke champion

Ganti teks `RUN_ID_MODEL` dengan run ID yang baru disalin:

```
bash
uv run python scripts/register_model.py --run-id "RUN_ID_MODEL"
```

Output registration menampilkan name, version, dan alias challenger. Review hasil model sebelum promotion. Script registration manual tidak otomatis memeriksa apakah model mengalahkan baseline. Ganti `VERSI_MODEL` dengan nilai version hasil registration:

```
bash
uv run python scripts/promote_model.py --version "VERSI_MODEL"
```

Jangan menganggap version selalu 1. Buka <http://localhost:8000/model-info>. **Hasil yang diperiksa:** HTTP 200, alias champion, serta version dan run ID yang sesuai. Setelah mengganti champion yang sudah di-cache, beri waktu sampai sekitar 30 detik untuk pengecekan registry berikutnya.

7. Kirim prediction traffic melalui HTTP

```
bash
uv run python scripts/generate_api_traffic.py \
  --date 2025-01-28 --start-hour 17 --end-hour 18
```

Hasil yang diperiksa: output menunjukkan request berhasil dan `data/monitoring/predictions.jsonl` mulai terisi. Script mengirim request untuk semua Manhattan zone pada jam 17 dan 18. API menggunakan hanya history sebelum masing-masing target hour. Karena ini replay, file actual pada target hour sudah tersedia untuk evaluasi sesudahnya. Tunggu 10-15 detik agar Prometheus melakukan beberapa scrape. Jika seluruh burst pertama selesai sebelum counter pertama di-scrape, panel rate/increase bisa masih nol. Jalankan command traffic sekali lagi setelah jeda tersebut.

Request berulang menambah counter HTTP, tetapi evaluasi memakai prediksi terakhir untuk kombinasi zone, target hour, dan model version yang sama.

8. Evaluasi dan buka Grafana

```
bash
uv run python scripts/evaluate_predictions.py
```

Hasil yang diperiksa: jumlah evaluasi lebih dari nol, recent MAE tersedia, dan artifact berikut terbentuk:

```
data/monitoring/evaluations.parquet
data/monitoring/performance_summary.json
```

Refresh dashboard Grafana setelah scrape berikutnya. Periksa:

- Request rate, prediction count, dan latency setelah HTTP traffic.
- **Serving Model (Last Prediction)** untuk versi yang menghasilkan prediksi.
- **Model MAE** untuk recent, reference, dan threshold.
- **Evaluation Samples** dan **Model Evaluation Status**.
- **Evaluated Model** untuk versi sumber MAE.
- Waktu job evaluasi dan target historical replay.

Status hasil evaluasi mengikuti data: sample kurang, dalam batas, atau perlu retrain. Hasil demo tidak harus selalu berwarna hijau.

Kalau masih kosong atau gagal

Gejala	Periksa	Tindakan
Docker tidak bisa terhubung	<code>docker info</code> dan context yang dipilih	Aktifkan engine dan gunakan context engine yang tersedia
Service belum siap	<code>docker compose ps</code>	Baca <code>docker compose logs --tail=100 mlflow api prometheus grafana</code>
<code>/model-info</code> memberi 503	Run ID, registration, alias, tracking URI	Register model HGB lalu promote ke champion pada server yang sama
<code>/predict</code> memberi 422	Isi response	Lengkapi history Jan 1-28, gunakan zone Manhattan dan target tepat awal jam
<code>/predict</code> berhasil tetapi graph kosong	Target Prometheus dan rentang waktu Grafana	Pastikan target taxi-api UP, pilih Last 30 minutes, ulangi traffic setelah scrape
MAE belum tersedia	Prediction log, versi model, dan actual demand	Jalankan evaluasi setelah prediction dan ground truth tersedia
Sample masih kurang	<code>evaluation_count</code> dan <code>min_samples</code>	Gunakan target hour/zone baru; request duplikat tidak menambah sample unik
Permission denied pada file mount	Pemilik <code>.mlflow/</code> atau <code>data/</code>	Sesuaikan izin folder untuk user app UID 1000; jangan menghapus state untuk menyembunyikan error

Setelah demo

Untuk menghentikan stack:

```
bash  
docker compose down
```

Data host dan named volumes tetap tersimpan karena command tidak memakai `-v`. Untuk demo berikutnya, jalankan service, kirim traffic baru, lalu evaluasi lagi. Panel **Predictions in 5m** menunjukkan window berjalan, sehingga wajar turun setelah tidak ada request baru.

Lampiran B - Troubleshooting

Kalau system banyak komponen, instinct beginner sering:

“Semuanya error.”

Padahal debugging lebih efektif kalau kita identify **boundary paling kecil yang fail**. Use checklist ini from bottom to top.

1. Setup layer

uv command not found

uv belum installed / PATH belum update.

Wrong Python

```
bash
uv run python --version
```

Expected 3.11.x.

Module missing

```
bash
uv sync
```

Airflow:

```
bash
uv sync --group airflow
```

Docs:

```
bash
uv sync --group docs
```

2. Data layer

TLC base URL AccessDenied

Normal kalau buka directory prefix. Use bootstrap script.

Historical file missing

Check exact date under data folders.

Feature build fail

Check processed demand completeness.

3. Model layer

Snapshot missing

Run DVC repro or manual snapshot creation.

Model worse than baseline

Not infrastructure bug. Inspect ML/features/model. Do not manipulate metric.

4. MLflow layer

UI unreachable

Is start_mlflow process running? Port 5000 free?

Registry empty after Docker transition

Workshop local + Docker should share .mlflow. Pastikan kalian pakai helper project dan Compose current config.

Champion missing

Promote registered version.

5. Airflow layer

Home Airflow terlihat kosong atau semua angka 0

Cek alamat dan instance dulu:

```
bash
docker compose ps airflow
```

Buka host port di kiri HOST:CONTAINER. Misalnya 127.0.0.1:18081->8080/tcp berarti gunakan <http://localhost:18081>. Jika membuka localhost:8080 saat Compose memakai 18081, kamu mungkin sedang melihat instance lain dengan riwayat berbeda. DAG workshop memakai manual trigger. Pilih DAG yang inputnya siap lalu tekan **Trigger**; sebelum run pertama, angka 0 adalah normal. Jika DAG tidak muncul, cek daftar DAG di instance yang sama:

```
bash
uv run --group airflow airflow dags list
```

Untuk instance Compose:

```
bash
docker compose exec airflow airflow dags list
```

Jika DAG tetap hilang, periksa log scheduler dan error import.

Task failed

Open task log. Ask:

```
input exists?
service reachable?
exception?
upstream output?
```

6. API layer

Connection refused

FastAPI process/container not listening.

/health 200, /model-info 503

API alive, model dependency unavailable.

/predict 422

Semantic input/history issue. Check target time and 168h history.

7. Docker layer

Port already in use

Stop local process.

API cannot reach MLflow

Check MLFLOW_TRACKING_URI. Inside Compose should point service hostname.

Data missing inside container

Check bind mount.

8. Monitoring layer

Prometheus target DOWN

Open:

```
http://localhost:9090/targets
```

Check API metrics reachable.

Grafana empty

Debug chain:

```
traffic?  
↓  
API metric?  
↓  
Prometheus scrape?  
↓  
query?  
↓  
Grafana datasource?
```

MAE panel empty

Need:

```
prediction log  
+  
ground truth  
+  
evaluate_predictions
```

9. CI layer

Branch policy fail

Check PR direction.

Ruff fail

Read exact file/line.

Docker build fail

Python tests green does not imply image build green. Read build logs.

Golden rule

Jangan random restart semua service setiap error. Itu bisa hide root cause. Tanya:

“Apa boundary pertama yang menghasilkan output yang salah?”

Then debug there. MLOps stack besar jadi manageable kalau kalian treat sebagai chain of smaller systems.

Lampiran C - Glosarium

Halaman ini buat quick reference. Kalau di tengah workshop ada istilah yang lupa, nggak perlu malu. MLOps memang banyak terminology.

Artifact

Output yang disimpan untuk dipakai atau diaudit nanti. Contoh:

- trained model,
- metrics JSON,
- Docker image,
- evaluation Parquet.

Baseline

Metode sederhana yang jadi pembandingan. Project kita:

```
prediction(t)
=
demand(t - 24h)
```

Model ML harus justify complexity dengan beat baseline.

Champion

Alias di Model Registry untuk model yang sekarang approved buat serving.

```
champion
→ version 3
```

Challenger

Candidate model yang sedang dipertimbangkan untuk replace champion. Belum automatically production.

CI

Continuous Integration. Automated checks saat source changes. Project kita check:

- lint,
- tests,
- docs,
- Docker builds.

CD

Bisa berarti Continuous Delivery atau Continuous Deployment. Project kita pakai **Continuous Delivery**:

```
main CI success
↓
publish container to GHCR
```

Belum auto-deploy ke cloud/server.

Container

Running instance dari Docker image.

```
image  
→ template  
  
container  
→ running instance
```

DAG

Directed Acyclic Graph. Airflow workflow representation.

```
task A  
↓  
task B  
↓  
task C
```

Directed = dependency punya arah. Acyclic = tidak ada circular dependency.

Data leakage

Feature memakai information yang belum seharusnya available saat prediction. Contoh buruk:

```
predict 18:00  
but feature includes actual 18:00 demand
```

Metric bisa terlihat bagus tapi fake.

DVC

Data Version Control. Di project ini dipakai untuk training snapshot pipeline reproducibility. Bukan replacement Git dan bukan experiment tracker.

Feature

Input untuk model. Contoh:

```
hour  
lag_24h  
rolling_mean_3h
```

Ground truth

Actual outcome yang dipakai buat evaluate previous prediction. Prediction 18:00 dibanding actual 18:00 setelah hour selesai.

Histogram

Prometheus metric type untuk distribution observation. Kita pakai buat API latency.

Idempotency

Rerun logical operation yang sama tidak menghasilkan unintended duplicate/corruption. Useful untuk pipeline retry/backfill.

Image

Built Docker artifact/template. Container dibuat dari image.

Inference

Proses menggunakan trained model untuk menghasilkan prediction.

Lag feature

Historical value pada offset tertentu.

```
lag_1h  
→ 1 hour before  
  
lag_24h  
→ same hour yesterday
```

Lineage

Trace dari output ke origin. Contoh:

```
prediction  
↓  
model version  
↓  
MLflow run  
↓  
dataset SHA
```

MAE

Mean Absolute Error. Average absolute prediction error. Kalau MAE 10:

Rata-rata prediction meleset sekitar 10 target units.

MLflow Run

Satu execution experiment yang punya:

- params,
- metrics,
- tags,
- artifacts,
- unique run ID.

Model Registry

Versioned catalog untuk model artifacts setelah experiment. Kita punya version + aliases.

Orchestration

Mengatur:

- step,
- dependency,
- schedule,
- execution state.

Airflow adalah orchestrator kita.

Prometheus Scrape

Prometheus melakukan HTTP request ke metrics endpoint untuk collect current values.

```
Prometheus  
→ GET /metrics
```

RMSE

Root Mean Squared Error. Regression metric yang memberi penalty lebih besar ke large errors.

Rolling Feature

Statistic dari recent historical window. Contoh:

```
rolling_mean_3h
```

average demand 3 completed hours sebelumnya.

Serving

Membuat inference available melalui stable interface. Project kita pakai FastAPI HTTP.

Snapshot

Frozen dataset state untuk satu training cycle.

```
taxi_demand_2025-01-26.parquet
```

Task

Unit of work dalam Airflow DAG. Contoh:

```
validate_batch
```

Training-serving consistency

Feature meaning saat training harus sama dengan saat serving. Kalau lag_24h semantics beda antara offline dan API, model quality bisa rusak.

Volume

Storage yang dipakai container dan punya lifecycle terpisah dari container process. Bisa Docker named volume atau bind mount depending context.

XCom

Airflow mechanism untuk passing small task metadata/result. Good:

```
date
path
row count
run ID
```

Bad:

```
huge DataFrame
```

Large data lebih baik lewat file/object storage.

Lampiran D - Laporan dan panduan fasilitator

Laporan hasil peserta

Setiap pasangan membuat satu laporan singkat dengan bukti berikut. Jangan menyalin angka contoh sebagai hasil praktik.

markdown

Hasil Workshop - Nama Peserta

Data dan experiment

- Revision kode yang dipakai:
- Snapshot dan SHA256:
- Periode train/validation:
- Run ID baseline / MAE / RMSE:
- Run ID model / MAE / RMSE:

Serving

- Champion model version dan source run ID:
- Contoh request dan response prediction:
- Hasil request invalid:

Monitoring dan keputusan

- Evaluation count / minimum sample:
- Recent MAE / reference MAE / threshold:
- Evaluation status:
- Waktu job evaluasi / latest replay target:
- Serving model / evaluated model:
- Tindakan yang disarankan beserta alasannya:

Reproducibility dan workflow

- Hasil dvc status:
- DAG yang dijalankan dan output bisnisnya:
- Satu keterbatasan implementasi dan usulan perbaikan:

Rubrik penilaian

Aspek	Bobot	Kriteria
Data dan leakage	20%	Menjelaskan cutoff, warm-up, dan batas history
Experiment dan lineage	25%	Membandingkan baseline dan menelusuri champion ke dataset
Serving	15%	Request berhasil, input invalid ditolak, metadata cocok
Monitoring dan keputusan	25%	Membaca sample, freshness, MAE, dan sumber versi
Orchestration dan CI/CD	15%	Menjelaskan task outcome dan membedakan lifecycle kode/model

Kelulusan praktik berarti evidence utama tersedia dan keputusan dapat dijelaskan. Model dengan MAE paling kecil bukan satu-satunya ukuran keberhasilan workshop.

Catatan fasilitator

Sebelum peserta datang

1. Jalankan panduan Grafana pada revision workshop yang akan dipakai, termasuk training, artifact reload, HTTP prediction, dan evaluasi dengan actual.
2. Catat run ID, model version, serta metric hasil preflight; hasil peserta dapat berbeda jika input/dependency berubah.
3. Periksa engine Docker, target Prometheus, dan dashboard provisioning.
4. Siapkan contoh CI run dan, jika ada, contoh challenger yang sudah direview.
5. Pastikan sesi Airflow daily replay Jan 27 lalu Jan 28 dapat berjalan. Terapkan workaround DVC container jika initial training DAG akan didemokan.

Saat mengajar

- Tanyakan alasan sebuah tool diperlukan sebelum membuka UI-nya.
- Berhenti di checkpoint bila model/data belum siap; UI service yang terbuka belum cukup menjadi bukti checkpoint inference.
- Bedakan hasil real peserta dengan angka ilustrasi dalam modul.
- Gunakan demo sample kurang dan request invalid untuk memperlihatkan status yang berbeda; tidak perlu memanipulasi actual agar dashboard terlihat bagus.
- Jika retrain tidak terpicu, baca summary dan gunakan fixture test untuk membahas cabang threshold. Jangan menjanjikan candidate baru untuk setiap DAG run.

Kunci diskusi singkat

Pertanyaan	Jawaban yang dicari
Mengapa warm-up tujuh hari?	Feature terjauh memerlukan lag 168 jam
Mengapa shift sebelum rolling?	Actual target tidak boleh menjadi feature
Mengapa baseline dan HGB memakai split sama?	Perbandingan model perlu konteks evaluasi yang sama
Mengapa Git dan DVC masih perlu MLflow?	Source/data state dan catatan experiment mempunyai tanggung jawab berbeda
Mengapa /health sukses belum menjamin inference?	Endpoint ini tidak memverifikasi champion dan kelengkapan history
Mengapa request berulang tidak menambah sample unik?	Evaluasi mempertahankan prediksi terakhir per zone/target/version
Mengapa retrain_recommended=false belum tentu model sehat?	Actual bisa belum ada atau sample belum cukup
Mengapa p95 rendah belum menjamin forecast baik?	Latency mengukur service; MAE memerlukan ground truth
Mengapa Serving Model dan Evaluated Model bisa berbeda?	Promotion dan job evaluasi berjalan pada waktu berbeda
Mengapa DAG sukses belum berarti candidate diregister?	Return bisnis dapat not_registered atau not_needed
Mengapa candidate baru belum langsung champion?	Baseline gate belum menggantikan review dan perbandingan champion
Mengapa publish image belum berarti deployment selesai?	Workflow repo menerbitkan image, tanpa rollout ke server

Batas implementasi dan latihan lanjutan

Workshop memakai filesystem lokal, SQLite metadata MLflow, Airflow standalone, dan Compose. Replay berjalan sesudah daily file tersedia; fitur dibatasi sebelum target, tetapi repo belum menyimulasikan latency kedatangan data secara real time. Monitoring model saat ini mengevaluasi error agregat terhadap threshold sederhana. Belum ada deteksi drift feature, alert routing, atau analisis kualitas per zone. Snapshot/lineage juga belum otomatis menyimpan semua informasi revision kode dan environment yang diperlukan untuk reproduksi lintas mesin.

Latihan lanjutan	Pertanyaan desain
Dependency yang terkunci dan digunakan image	Apakah host, CI, dan container memasang versi yang sama?
Validasi grid offline	Bagaimana mendeteksi duplicate/missing hour sebelum shift baris?
Jadwal dan dependency antar DAG	Bagaimana memastikan evaluasi menunggu actual tersedia?
Retraining idempotent	Bagaimana retry/run berulang tidak membuat candidate duplikat?
Readiness inference	Apa yang perlu dicek selain proses API hidup?

Latihan lanjutan	Pertanyaan desain
Alert dan freshness	Kapan operator diberi tahu tentang scrape gagal atau evaluasi lama?
Evaluasi per zone dan common holdout	Apakah kualitas agregat menyembunyikan area yang buruk?
Storage dan model review bersama	Bagaimana dataset, artifact, dan approval dibagikan antar tim?

Pilih satu latihan, jelaskan perubahan yang diusulkan, dan tentukan bukti yang akan dipakai untuk memverifikasi perbaikannya.